AMAZON L6 · SENIOR SUPPLY CHAIN MANAGER · NACF SUPPLY CHAIN · PACKET 3B (COMPLEMENT TO THE ROCKY PROGRAM-MANAGEMENT PACKET)

The Bar Raiser Bank — Rocky Goforth

More examples, all 16 Leadership Principles, three-level probes, failures — every answer in STAR

Rocky Goforth — Senior Manager, Amazon Infrastructure · BAR RAISER

Wants an engineering / program leader: large-scale deployment, program cadence, stakeholder management, cost-versus-speed calls

Friday Oct 30 · 10:00 am PDT · first interview of the day

CANDIDATE: RAJESH RISHIYUR CHANDRASEKARAN · BUILT THU OCT 29, 2026 · READ WITH: AMAZON-NACF-PANEL-3-GOFORTH-PGM-STAR-KIMI-K3 (PRIMARIES P1–P7)

The one-line strategy. Rocky is the bar raiser, so he will not stay on one story. He will pick a Leadership Principle, ask a question from Amazon's own bank, and dig three levels: *what exactly did you do · what did the data say · what happened after*. The program packet (P1–P7) gives him the big UDC and WMS programs. This bank gives him **nine more real stories (R1–R9)** and **four real failures (F1–F4)** that were not told to the hiring manager or Ryan and are not the stories Venkata will hear. That way the loop never hears the same story twice.

Order to reach for them. P1 UDC launch → R1 routing guide (25 European locations) → R5 observability on a nine-team platform → R6 two programs, one ruler → R2 carrier-bid cycle → R4 Customs ownership split → R7 the triage agent → F1 the UAT that crashed (only when he asks for a failure).

Three opening sentences. (1) "The noise said the carriers were late; three years of shipment data said we could not even measure late. Data quality was about 55%, not the 40% on the report, so I fixed the measurement before the engine." (2) "Six systems and nine teams, and no ID tied an event to its source, so every blocker took two to four hours to find. I proposed one trace ID end to end, and the time to fix a blocker bug fell by half." (3) "Leadership pulled a go-live forward a month and moved my engineers onto it, so I ran both programs on one RAID log, one 45-minute daily forum and one blocker ruler, and both landed."

Five numbers never to get wrong.

1. **About $8M validated freight savings across 25 European locations** (record, Apple F2). The Austria pilot was about $500K a year at one site. Never say "$1M", never say "400-plus lanes".
2. **Data quality: about 40% reported, about 55% true, then 99%**, "within a quarter". Apple says two months and the master says three, so I say "within a quarter".
3. **Time to fix blocker bugs down 50%**. It used to take two to four hours just to find which system held the bug, across six systems and nine teams (record-from-docs, SV 3PL ingestion one-pager).
4. **$25 to $0.40 per triaged incident, a 98% cut, with 100% citation coverage** (record, Apple I1). This was my own independent build, not an employer program.
5. **About $200K a month per DC was at stake** in the compressed go-live (record, Apple B3). The compressed program landed inside four weeks. Quote the monthly rate first and "about $1M per DC" only if asked.

Sources: master packet, GFiber v8, Apple PM III v11, Flexport and UDC program docs, AIQB. Full SOURCE TRACE at the back. Basis labels: **record** · **record-from-docs** · **user anchor** (GFiber v8 only) · **modeled** · **illustrative**. No detailed math tables.

1 · Story inventory & allocation — every real story, who already heard it, what is FREE for Rocky

Legend: **Told** = told today to Snigdha Tripathy (HM) or Ryan Hoerger · **P#** = primary in Rocky's program packet · **Josyula** = likely in Venkata's 12:00 bank (SV/Crux as product, S9, S13–S16, Walmart design cases, failure arcs) · **FREE** = nobody has it, so use it first. Any reuse is told from a *different angle with different facts* and flagged. **Net:** six FREE (R1, R2, R3, R4, R6, R7), three different-angle reuses (R5, R8, R9), four failures (F1–F4). No design case needed. The UDC arch-council deck and the Canada bulk-ship discovery go to the separate UDC/Canada story pack.

STORY (SOURCE CHAPTERS)	COMPANY	TOLD TODAY TO HM / RYAN?	ROCKY PM PACKET?	LIKELY JOSYULA?	FREE? → USE HERE
S1 / E1 / S3 — S/4HANA ↔ ISM "available"; Shipped-Not-Billed	Sephora	Told (HM)	—	—	No — one-line bridge only
S2 / C1 — UDC dual-inventory comingle launch	Sephora	—	P1	—	PM packet; bank answers DR Q1
C2 — UDC issues log + build plan (836 rows)	Sephora	—	inside P1 (numbers)	—	**R8 — the mechanism angle (log as readiness gate)**
C3 — Locus robotics gate (98% bar, week-2 94%)	Sephora	—	P3	—	PM packet; bank answers HSS Q1
Locus retail-testing (RTL) slip, three weeks	Sephora	—	mentioned in P1/P3	—	**F4 — told only as a failure**
S4 — in-transit promise rule I got wrong	Sephora	In Ryan's router as his failure answer — **check whether he heard it**	—	possible (failure arcs)	**F3 — use only if Ryan did not hear it; otherwise use F1**
S5 / C4 — architecture council, three decisions	Sephora	—	P5	—	PM packet
S6 / F3 — shortage alert, $2M released	Elementum	Told (safety stock)	—	—	No
S7 — six-site contract-manufacturer rollout	Elementum	Ryan's bench only (not a primary)	—	—	**FREE → R3 (site grain, three waves)**
S8 / F4 — 200-lane lead-time distributions	Elementum	Told (200 lanes)	—	—	No
S9 / F1 — a thousand lanes, phased by value	Elementum	Ryan primary (may have been told)	—	Josyula	No
S10 / F2 — routing-guide engine, Austria → 25 locations	Elementum	—	—	—	**FREE → R1**
S11 / A1 / A2 / A5 — SV wave machine, WBR, launch gate	Flexport	Told (HM)	—	Josyula (product)	No as told
SV 3PL ingestion one-pager + E2E connectivity monitoring	Flexport	HM heard only the "monitoring came late" lesson	—	—	**R5 — the infrastructure angle (trace ID, monitors, kill switches)** **SV FAMILY — DIFFERENT ANGLE**
A3 / S16 — four systems → one Snowflake pipeline	Flexport	In HM packet (S16-A)	—	Josyula (S16)	Only its failure G3 → F2 **ALSO WITH JOSYULA — DIFFERENT ANGLE**
A4 — Crux decommission (79 customers)	Flexport	In HM packet (S11-C)	P4	Josyula	No
S12 / B1 / B2 / S12-A — FF App roadmap flip, CTS math, Asia latency	Flexport	Told (estimation)	—	—	No
B3 — compressed go-live, two programs on one ruler	Flexport	—	—	—	**FREE → R6** **FF APP FAMILY — DIFFERENT ANGLE**
Customs Platform ↔ FF App ownership proposal	Flexport	—	—	—	**FREE → R4**
S13 / M1 — Cyber Monday 4 AM	Manhattan	—	P6	Josyula (S13)	Bank answer BfA Q4 only (P6 lens)
S14 — the $2.5M-of-order-value discrepancy, told the customer	Manhattan	—	P6	Josyula	No
S15 / D3 — 20-person team to a four-week cadence	Sephora	—	—	Josyula (S15)	**R9 — the develop-people angle (QA lead owns the gate)** **ALSO WITH JOSYULA — DIFFERENT ANGLE**
S17 — semi-annual carrier bid, 6 → ~3 weeks	Elementum	—	—	—	**FREE → R2**
S18 / D1 / D2 — RP → JDA WMS migration	Sephora	—	P2	—	PM packet; bank answers L&BC Q1, DR Q2
G1 — the UAT that crashed (Apple Part 6)	Elementum	—	—	possible	**FREE → F1**
G2 — RF-device script (Apple Part 6)	Sephora	—	P7 (likely)	possible	Left to P7; bank answer ET Q4 points there
G3 — security review discovered late (Apple Part 6)	Flexport	—	—	possible	**F2** **ALSO WITH JOSYULA — DIFFERENT ANGLE**
I1 — LLM triage agent, $25 → $0.40	Independent	—	—	—	**FREE → R7**
S19–S21 Walmart · E1–E12 GFiber design rounds	Design cases	Ryan (S19–S21)	—	Josyula	Not used — real stories cover every LP

2 · Bar Raiser playbook NEW

What a Bar Raiser is actually testing

LP depth, not LP coverage	One story, three levels down. A story that only works at level one counts as a "mild concern" in the AIQB rubric. The proof is a mechanism you can name (the gate, the ruler, the log), plus the number it moved and the date.
Ownership vs "we"	Every sentence that carries a claim starts with "I". Every sentence about the build names who built it. He will ask "what exactly did *you* do?" at least twice, so answer with an artifact you wrote: the rules spec, the RAID log, the trace-ID proposal, the eval set.
Data-driven honesty	The basis word goes with every number: record, from the doc, my estimate, modeled. A Bar Raiser rewards "I don't know, and here is how I would find out" over a confident decimal.
Raising the bar	Would this person raise the average of the team? Show that standards were set *before* the pressure arrived (the 99% data gate, the <5% mismatch gate, the 98% pick bar), and that a control outlived the program.
Long-term thinking	Things I gave up in the short term for the long term: the engine waited for the data, the cheap path was proved on an eval set first, and the access review now sits on every kickoff checklist.
Self-awareness	A real failure, owned in the first sentence, with the root cause in my own process and a standing control that proves the lesson stuck.

How the three-level dig goes, and the pre-emptive line for each level

Level 1, "What exactly did YOU do?" Pre-empt it inside the Action: "I wrote… I ran… I decided…; engineering built…; Finance signed…". End the Action with one sentence that separates *mine* from *theirs*.

Level 2, "What was the data? How did you know?" Pre-empt it with the baseline number, the measurement grain and the source in the Situation: "three years of shipment history I pulled in SQL", "836 rows, March 2024 to October 2025".

Level 3a, "Who disagreed, and were they right?" Name a real dissenter and concede the part where they were right: the procurement team I brought in late, QA's fear of compressed testing, leadership pulling my engineers.

Level 3b, "What would you do differently?" One concrete change in my own process. It is never "communicate more". Examples: build the audit ledger during the pilot, version the eval set on day one, ship the quick-reference guide earlier.

Level 3c, "What happened after?" The control that outlived me: Finance reused the savings method, other regions adopted the tool without a mandate, the trace ID became the standard way to debug.

Pre-empt technique (say it once, early): "I'll tell you what I owned, what the team built, and the number with its source. Stop me where you want depth." After that, every answer runs Situation with baseline, Task with my scope, Action with my artifacts, Result with the number and its basis, then one line of what I'd do differently.

"I vs we" ledger — the five most attackable claims

CLAIM	WHAT WAS MINE	WHAT WAS THEIRS	SENTENCE IF HE PUSHES
About $8M validated freight savings (R1)	The SQL baseline, the rules-engine spec, the data gate, the savings method co-written with Finance, the pilot read	Carriers and the transportation team executed the tenders; Finance audited the claims; engineering built the engine	"The $8M is executed savings, counted per shipment by a method Finance co-owned before the first claim. I owned the method and the gate, not the freight contracts."
Time to fix blocker bugs down 50% (R5)	Noticed the missing ID while reading application logs in the bug bash; proposed the design change; defined the load-event tracing metric; ran the monitoring checklist	Engineering on the Visibility, Ocean and Flexport OS teams implemented the trace stories and the Datadog monitors	"I found it and proposed it; three teams built it. The 50% is from my one-pager, measured on blocker bugs."
Both programs landed in the compression (R6)	The RAID log, the 45-minute forum, the blocker ruler, escalations framed as priced decisions, the quick-reference guide	Shared engineers did the build and both war-room passes; the DC floor adopted	"Making the collision visible and survivable was mine; the code and the adoption were theirs."
Carrier-bid cycle 6 weeks → about 3 (R2)	The time audit, the lane-master spec, the constraint rules, the <5% gate, the parallel run	Transportation and procurement ran the award; engineering built the tool	"Cycle time is the number I own. I claim no award savings, because the award was theirs." (user anchor)
$25 → $0.40 per incident (R7)	All of it: the code, the eval set, the routing, the citation rule	Nobody else, because it was an independent build	"This one is mine end to end, which is why I tell it as a build story and not as a program story."

16-LP self-audit — my strongest evidence line, its number, and the soft spot I name first

LP	STRONGEST EVIDENCE LINE	NUMBER (BASIS)	SOFT SPOT TO NAME FIRST
Customer Obsession	R1: everyone asked for "fix the carriers", but the real need was "make lateness measurable"	DQ ~40% reported, ~55% true → 99% (record)	F1: I once defended with screenshots instead of reproducing the customer's experience
Ownership	R5: as TPM I read application logs in the bug bash and proposed a cross-team design change	MTTR −50% (record-from-docs)	F2: I discovered the security review instead of planning it (two weeks)
Invent & Simplify	R2: a reconciliation problem everyone called an optimization problem	6 weeks → ~3 (user anchor)	Procurement joined late, and that was my miss
Are Right, A Lot	R5: a production test account instead of staging, decided on pros and cons with product and engineering	300-container start (record-from-docs)	F3: my first in-transit rule was too conservative
Learn & Be Curious	R7: I learned LLM engineering by building an agent, not by reading	$25 → $0.40 (record)	The eval set was not versioned on day one
Hire & Develop	R9: the QA lead authored the definition of done; R3: site super-users own the mapping tables	4-week cadence in 6 weeks (record)	No large direct-report org, and I say so (Apple gap script)
Highest Standards	P3: the 98% pick bar, set before the robots arrived; R8: one log, one grammar	Week 2 at 94% → 30% throttled (record)	Should have put the monthly shape on the wall from month one
Think Big	R1: one Austria pilot became a method for 25 locations, and Finance reused it	~$8M across 25 European locations (record)	The audit-ledger format settled late
Bias for Action	R6: two programs on one ruler, decided inside week one	~$200K/month per DC at stake (record)	The team's calendar paid for the compression
Frugality	R7: cost engineered like a budget line, with accuracy held	−98% per incident (record)	At Amazon scale, the eval-set upkeep becomes a program of its own
Earn Trust	R4: one owner per work item between two orgs, written down and agreed	15 work items for 2023 (record-from-docs)	ISF filing stayed unresolved at the first meeting; my authorship is VERIFY
Dive Deep	R5: I created the load-event tracing metric	2–4 hrs to find a bug's source (record-from-docs)	Monitoring arrived later than it should have (Path to 4PL lesson)
Backbone	R6: I pushed for visibility, not against the decision, then committed	Landed inside 4 weeks (record)	I did not win the argument over engineers, and I say so
Deliver Results	P1: combined go-live with zero customer-facing incidents; P2: eight months, 3% under budget	+15% processing efficiency (record)	Training slipped 1.5 weeks on JDA (P7)
Earth's Best Employer	R6: a two-pass war room so nobody worked 24 hours; R9: QA's fear became the design	45-minute forum (record)	Thin. I don't stretch it into a DEI story I don't have
Broad Responsibility	R4: shared cost-to-serve goals across two orgs; F3: I changed a rule that was hurting customers	Oversell increase = zero (user anchor)	The in-transit rule starved dotcom for weeks before I fixed it

3 · More examples — nine additional full-STAR stories (R1–R9) NEW

None of these are P1–P7. Each story has a 60-second pitch, S/T/A/R, who I influenced, TRADE-OFF I MADE, a failure/learning line, LP tags, four probing Q&As, SAFE-SAY and a 20-second version. Program-leader framing throughout, with numbers in prose and their basis labels.

R1 · Austria first, then 25 European locations — the routing-guide engine waits for the data FREE

Company: Elementum (supply-chain SaaS), client logistics program · **My role:** program lead for the analysis, the rules design and the rollout gate · **Sources:** Apple v11 F2 p35–37; GFiber v8 S10 p120–121; master S10
Headline: about $8M validated freight savings across 25 European locations (record) · Austria pilot about $500K a year (record) · data quality ~40% reported / ~55% true → 99% within a quarter (record)

60-SECOND PITCH

"At Elementum, a client's transportation team in Austria could not measure on-time delivery. The report said data quality was 40%, the team spent eight to ten hours a week reconciling dates by hand, and everyone believed the carriers were late. I owned the analysis, the rules design and the rollout gate. I pulled three years of shipment history in SQL myself. True data quality was about 55%, and the problem was measurement, not carriers. I set one rule: the routing engine waits for the data. Nothing scales until quality hits 99%. I also wrote the savings method with Finance before anyone claimed a dollar: contracted rate against what would have been paid, per shipment, executed only. The Austria pilot went live inside a month, measured about 97% on-time accuracy and about $500K a year, and that green-lit 25 European locations. It closed at about eight million dollars of validated freight savings, and Finance reused the method afterwards."

S — Situation. The client's Austria site had a routing guide on paper and no enforceable on-time delivery (OTD). Reported inbound data quality was about 40%, and logistics spent 8–10 hours a week reconciling promised dates against actual dates (record, Apple F2). The site, the carriers' account teams and my own sponsor all believed the carriers were the problem. The freight base had no SLA anyone could hold a carrier to.

T — Task. I owned the analysis, the rules-engine design, the savings-validation method and the scale gate. Transportation, Finance and engineering did not report to me. The targets were set with the business: data quality to 99%, the manual reconciliation retired, and every savings dollar auditable.

A — Action.

- **Baseline first.** I pulled three years of shipment history in SQL and ran a defect census. The defects were missing contracted rates, lanes keyed at the wrong grain, carrier names in five spellings, and transit times copied from a brochure. True quality was ~55%, not 40%. I took the reframe to the sponsor: we are not measuring lateness yet.
- **Method before money.** I co-wrote the savings definition with Finance before the first claim: contracted guide rate minus counterfactual paid rate, per shipment, per lane, *executed* savings only, never "negotiated".
- **The gate.** The engine waits for the data. The quality gate was 99% before scale, tracked weekly by defect class, with each class owned and burned down on the same weekly read.
- **Rules written down before configuration.** Ship-from, ship-to, mode, volume, Incoterm and SLA. A lane with no match pages a human; it never falls back to a default carrier. Gaps after go-live surfaced as alerts with reason codes.
- **Pilot on the ugliest mix, then scale off the pilot's own numbers.** Austria went live inside one month. Super-users per region and a one-page runbook carried the rollout to 25 European locations (runbook and super-users:

user anchor, GFiber v8).

R — Result.

- About **$8M of validated freight savings across 25 European locations**. Every dollar is auditable back to a shipment (record, Apple F2 and resume).
- Data quality **~55% true → 99%**, "within a quarter" (record. Apple says two months, the master says about three, so the quarter is true under both).
- Austria pilot: **~97% on-time measurement accuracy and ~$500K a year at one site**. Those were the numbers that made the scale call (record).
- Manual reconciliation of **8–10 hours a week** retired (record). The "1–2 hours after" figure is a user anchor (GFiber v8), and I say it only if asked.
- Finance **reused the validation method** for the next claims, so the control outlived the program (record).

Who I influenced: the sponsor (from "carriers are late" to "we can't measure late"), using his own three years of data · Finance (co-owner of the savings definition before any claim) · transportation managers (from picking carriers to managing exceptions with reason codes) · regional super-users (they own the rules files).

TRADE-OFF I MADE

Options: (a) turn on the engine now and clean the data in flight, or (b) hold the engine until quality reached 99%. I chose (b). I rejected (a) because an engine fed dirty lanes tenders with confidence and gets it wrong, and one visibly wrong tender costs more trust than a month of delay. It cost a slower start and savings deferred by weeks, and the sponsor had to wait for the first dollar.

Failure / learning: The audit ledger's format settled while the first claims were already being made. I would build it during the Austria pilot, which would have made the first CFO review shorter.

Dive Deep — three years of history in SQL showed 55% true, not 40% reported.

Think Big — one site's pilot was built as the template for 25 locations.

Insist on the Highest Standards — the 99% gate held before scale.

Customer Obsession — the customer asked for "fix the carriers" when the need was "make late measurable".

PROBING — FOUR QUESTIONS HE WILL ASK

"$8M — how do you avoid double counting?"

Each saving is counted once, per shipment, as the contracted guide rate against the counterfactual paid rate, executed only. Double counting lives in programs that count "negotiated" and "executed" separately. Finance co-owned the definition before the first claim.

"What exactly did you do versus the engineers?"

I pulled the data and ran the census, wrote the rules spec and the gate, co-wrote the method, and read the pilot to make the scale call. Engineering built and configured the engine. The carriers' performance was the carriers'.

"Who disagreed?"

My sponsor and the site, at first. They wanted carrier penalties now. They were right that carriers had real misses, and the scorecard caught those later. They were wrong that we could prove it on 55%-quality data. The SQL census settled it, so I didn't have to argue.

"What happened after you left the program?"

Finance kept the validation method for the next claims, and the super-users kept the rules files. A new site cost one rules file and one data-quality cycle (user anchor). That is the mechanism, not me.

SAFE-SAY (most attackable number): "About eight million of validated freight savings across twenty-five European locations, counted per shipment, executed only, by a method Finance co-owned. That is the record figure, and I never say one million or a lane count."

"What's your source?" "My program record and resume: the Austria pilot read and the Finance-validated ledger."

20-SECOND VERSION

"Austria said carriers were late. Three years of data in SQL said we couldn't even measure late: 55% true quality, not 40%. I held the engine until data hit 99% and wrote the savings method with Finance first. The pilot was 97% accurate and worth about $500K a year, and the rollout to 25 European locations closed at about $8M validated."

R2 · The semi-annual carrier bid — a reconciliation problem everyone called an optimization problem FREE

Company: Elementum, client transportation team · **My role:** owned requirements, constraint rules, validation gate and parallel run, not the award · **Sources:** GFiber v8 S17 p166–167; master S17
Headline: bid cycle 6 weeks → ~3; pilot region 3 weeks early; adopted by other regions without a mandate (all **user anchor** — GFiber v8 only; no Apple or primary-doc support). No dollar claimed.

60-SECOND PITCH

"A client ran a carrier bid twice a year: hundreds of lanes, dozens of rate files, six weeks a cycle. Everyone assumed it was an optimization problem and wanted a better solver. Before building anything, I sat through one full bid cycle and logged where the hours went. Most of the six weeks was reconciliation, because every carrier named lanes and modes differently. So I specified a lane master, wrote the constraints as explicit rules, and set a gate: the tool had to match the last two real awards within 5% before anyone trusted its output. We parallel-ran it on one region's live bid. The cycle went from about six weeks to about three, the pilot region finished three weeks early, and other regions adopted it without a mandate. My miss: procurement joined late."

S — Situation. The client's transportation team ran a semi-annual carrier bid across hundreds of lanes and dozens of carrier rate files, with constraints like incumbency floors and volume commitments. Each cycle took about six weeks (user anchor). The team's working belief was "we need a better optimizer".

T — Task. I owned the requirements, the constraint rules and the parallel run. Transportation, procurement and engineering did not report to me, and the award itself was theirs.

A — Action.

- **Diagnose the problem class with a time audit.** I sat through a whole bid cycle logging hours by activity. The six weeks went to reconciling lane names and mode codes, not to analysis. I wrote a one-page problem-class memo: reconciliation, not optimization.
- **Simplify the input, not the solver.** A lane-master spec normalizing origin, destination and mode. Every constraint became one explicit, readable rule: volume commitments, incumbency floors, mode rules.
- **A trust gate before any output counted.** Less than 5% mismatch against the last two award cycles before the optimizer could run. Scenario comparison was a first-class output, and I rejected a black box in writing.
- **Parallel run on a live bid.** One regional team's real bid ran on both paths.
- **Keep it honest after cutover.** Weekly rate-file refresh, with a stale-data banner after 10 days.

R — Result.

- Bid cycle **about six weeks → about three** (user anchor). The pilot region finished **three weeks early** (user anchor).
- Other regions **adopted it without a mandate** (user anchor).
- **No award dollar claimed.** The award belonged to transportation and procurement. Any dollar figure for analyst time returned is modeled only, and I don't lead with it.

Who I influenced: transportation leadership (from "buy a better solver" to "fix the lane names", using their own hours) · the regional team that volunteered the parallel run · engineering (scenario comparison instead of a black box) · procurement, late, and that is the miss.

TRADE-OFF I MADE

Options: (a) build or buy a stronger optimizer, which everyone expected, or (b) fix reconciliation first and keep the solver simple and explainable. I chose (b). I rejected (a) because a better solver on mismatched lane names gives a faster wrong answer. It cost one full bid cycle of observation before any build, and some impatience from a team that wanted a tool.

Failure / learning: Procurement joined late. They owned award rules I had to retrofit. Lesson: put every owner of a constraint in the room at discovery, not at design review.

Invent & Simplify — the usual approach (a better solver) wouldn't address it; a lane master would.

Dive Deep — a time audit across one full cycle, before any design.

Have Backbone — I said "it's not an optimization problem" to a team that had already decided it was.

Deliver Results — about half the cycle time, with the pilot three weeks early.

PROBING — FOUR QUESTIONS HE WILL ASK

"How do you know the time saved wasn't just a quieter cycle?"

The same team on the same kind of bid, measured against my own time audit of the previous cycle. The honest caveat: those counts are my anchors from the program and not an audited report, so I say "about half".

"Why a 5% gate?"

It had to beat a planner's gut on the last two awards they already trusted. Below 5% mismatch, the planners trusted the tool enough to stop re-keying. Above it, they would have kept running the old spreadsheet beside it.

"Who disagreed, and were they right?"

Procurement, when they arrived. They were right that two award rules weren't in my constraint set, so I added them as explicit rules before the second region.

"What would you do differently?"

Bring procurement in at the time audit. I'd also publish the stale-data banner rule from day one, not after the first stale week.

SAFE-SAY: "About six weeks to about three. Those are my program anchors, and I claim no award savings because the award belonged to transportation and procurement."

"What's your source?" "My own time audit of the bid cycle and the program record. There's no published report, so I give you the shape and the gate."

20-SECOND VERSION

"Everyone wanted a better carrier-bid optimizer. I sat through a full cycle and found the six weeks were reconciliation, not optimization. I built a lane master, wrote the constraints as rules and set a 5% match gate, and the cycle went to about three weeks. My miss was bringing procurement in late."

R3 · Six sites, three waves — a contract manufacturer's data rolled out site by site, with reconciliation as the go-live gate FREE

Company: Elementum, client manufacturing program with an external contract manufacturer · **My role:** owned ingest requirements, the reconciliation gate and the wave plan, and was the person on the weekly call with the contract manufacturer · **Sources:** GFiber v8 S7 p93; master S7 (told here at site grain)
Headline: 0 → 6 sites with planning-grade supply visibility in three waves over about five months; hardest site took 6 weeks against a 3-week average and was absorbed in the plan; zero overlapping revisions auto-loaded (all **user anchor** — GFiber v8 only). No dollar claimed.

60-SECOND PITCH

"At Elementum, a client's supply visibility depended on data from an external contract manufacturer across six site systems. The six sites had six different definitions of yield, lead time and order status. I owned the ingest requirements, the go-live gate and the wave plan. I sequenced it like a deployment. Wave one was the two cleanest high-volume sites, to prove the pattern. Wave two was three sites on parameterized maps. Wave three was the hostile one: a legacy system with flat files and no change-data-capture. The rule: no site goes live until its reconciliation report matches the source. Overlapping revisions went to a quarantine queue for a human to decide, and were never auto-loaded. A named super-user at each site owned the mapping tables. Six sites went live in three waves over about five months. The hostile site took six weeks against a three-week average, and that was absorbed in the plan because I had sequenced it last."

S — Situation. The client's supply view depended on data an external contract manufacturer published from six site systems. The six systems carried six definitions of yield, lead time and order status. Two revisions of the same data could be partly valid at the same time for weeks, so auto-loading one would silently rewrite history. One site ran a legacy system with flat-file exports and no change-data-capture (user anchor).

T — Task. I owned the ingest requirements, the per-site reconciliation gate, the wave plan and the weekly call with the contract manufacturer. Their engineers, our data engineers and the client's planners did not report to me.

A — Action.

- **Catalogue before you map.** I built a six-site definition inventory (yield, lead time, order status) and a discovery memo naming the traps, and I had the contract manufacturer's engineering confirm it.
- **Wave by risk, not by geography.** Wave 1 was the two cleanest high-volume sites. Wave 2 was three sites on parameterized maps. Wave 3 was the hostile legacy system, scheduled last with buffer.
- **A gate that can stop a site.** A site went live only when its reconciliation report matched the source within tolerance. The tolerance value is not in my record, so I don't quote one.
- **Quarantine, never auto-load.** Overlapping revisions went to an effective-date quarantine queue for human disposition.
- **Engineer around the hostile system.** With no change-data-capture, we used snapshot-diffing on the flat files instead of waiting for an upgrade the site couldn't fund.
- **Hand ownership to the sites.** A named super-user at each site owned that site's mapping tables. The weekly call with the contract manufacturer became standing, framed as their protection, not our audit.

R — Result.

- **0 → 6 sites** with planning-grade supply visibility, in **three waves over about five months** (user anchor).
- **Hardest site 6 weeks vs a 3-week average**, absorbed because it was planned last with buffer (user anchor).
- **Zero overlapping revisions auto-loaded** (user anchor). This data backbone is what the later lead-time program ran on.

- No dollar claimed. Anything priced here would be modeled, and I don't lead with it.

Who I influenced: the contract manufacturer's engineering (confirm our reading of their data; the weekly call as their protection) · six site leads (each named a super-user to own the maps) · our data engineers (snapshot-diffing instead of waiting for change-data-capture) · the client's planners (no site counted until it reconciled).

TRADE-OFF I MADE

Options: (a) a big-bang load of all six sites on one generic map, or (b) three waves with per-site reconciliation gates. I chose (b). I rejected (a) because one bad site would have poisoned trust in all six. It cost about five months instead of a faster single cutover, and the hostile site's six weeks had to be carried openly.

Failure / learning: I learned to reconcile to the source's actual data, not its documentation. The documentation was not the truth; the source system's own numbers were. Next time I'd run the reconciliation report on a sample before writing the mapping spec, not after.

Deliver Results — six sites in three waves; the obstacle (no change-data-capture) was planned for, not discovered.

Frugality — snapshot-diffing instead of funding a legacy upgrade.

Hire & Develop the Best — site super-users own their maps; the capability lives at the site.

Insist on the Highest Standards — no site live until it reconciled.

PROBING — FOUR QUESTIONS HE WILL ASK

"What was the tolerance on the reconciliation gate?"

It isn't in my record, so I won't give you a number. The design was a per-site report that compared our loaded view against the contract manufacturer's own system for the same items and dates, and the site waited until it matched.

"Why not make the legacy site upgrade?"

No budget and no timeline. Snapshot-diffing the flat files gave us change detection without asking them to fund anything. It cost three extra weeks, which the plan already carried.

"What did you personally do on that hostile site?"

I sequenced it last, wrote the requirement for snapshot-diffing, and ran the weekly call where its reconciliation was read. Data engineering built the diff.

"What happened after?"

The super-users kept the maps, the weekly call stayed standing, and the lead-time distribution work (told elsewhere) ran on this backbone.

SAFE-SAY: "Six sites in three waves over about five months. Those are my program anchors, and I give no dollar figure because the record holds none."

"What's your source?" "My program record of the wave plan. The counts come from me, not from a published report."

20-SECOND VERSION

"An external manufacturer's data across six site systems, six definitions of the same fields. I rolled it out in three waves by risk, with reconciliation as the go-live gate and super-users owning each site's maps. Six sites in about five months, and the hostile site's extra three weeks were already in the plan."

R4 · One owner per work item — the Customs Platform ↔ FF App ownership split FREE

Company: Flexport, 2022, Freight Forwarding App (FFA) export program · **My role:** FF App-side program lead on the ownership split VERIFY: role and authorship — my name is not on either meeting note's attendee list · **Source:** Customs_Platform_Ownership_Proposal p1–14 (meeting notes Oct 26 and Nov 3, 2022)
Headline: 15 work items planned for 2023 (12 export, 3 import) (record-from-docs) · agreement on Nov 3, 2022: FFA owns all export work items except ISF filing; AMS and AES through Customs Platform APIs; shared cost-to-serve goals (record-from-docs).

60-SECOND PITCH

"At Flexport in 2022, two tech orgs, Customs Platform and the Freight Forwarding App, were both building pieces of export customs. 2023 had fifteen work items on the plan, twelve export and three import, and nobody could say who owned which. That meant duplicate builds and two teams blocking each other. I took it as a design problem, not a turf fight. Customs owns customs planning and execution; the forwarding app owns the operational work of moving cargo onto a vessel. I put every work item in one table with its user group, whether it already existed in core, the proposed owner and the open question, and we worked it row by row in two sessions. By November 3rd: the app owns every export work item except ISF filing; AMS and AES go through Customs Platform APIs; and both teams share the cost-to-serve goal. The split was in writing before a line of 2023 code."

S — Situation. In 2022 the Customs Platform and FFA tech orgs overlapped on customs declaration work, especially the AMS (automated manifest) work item. The 2023 plan held **15 work items: 12 export and 3 import** (record-from-docs, p1). With no owner per item, there was a risk of duplicate builds, cross-team blocking and fuzzy accountability for cost-to-serve.

T — Task. Get one named owner per work item, agreed by both orgs, before 2023 build planning. Neither org reported to the other, and neither reported to me.

A — Action.

- **A principle first, then the rows.** Customs Platform focuses on customs planning and execution. FFA focuses on the operational execution of moving cargo onto a vessel, including the backend services, data models and automation for its own items.
- **One table, every work item.** Columns: work item, user group, description, exists in core?, proposed owner, notes, engineering conclusion. Upload SLI, File AES, Request/Generate/Upload EAD, portfiling, China export documents, ISF worksheet and filing, AMS.
- **Decide on qualification, not preference.** Only licensed brokers can file AES and ISF, so those stayed with customs or used its APIs. Portfiling is a seaport requirement tied to the EAD, so it sat with operations.
- **Integration contracts instead of shared code.** AMS: Customs Platform exposes atomic APIs (create, amend, query, delete). FFA collects the filing data and monitors for changes that need a re-file. Customs translates disposition codes and broadcasts status.
- **Write down the "not now".** EAD stays offline with external brokers until customs can integrate with them. China customs documents may be revisited in 2025. Import items come next, API-only.

R — Result.

- Final export agreement on **Nov 3, 2022**: **FFA owns all export work items except ISF filing** (record-from-docs, p14).
- **AMS and AES through Customs Platform APIs**, so integration replaced duplicate builds. Customs Tech and FFA **share cost-to-serve goals** on AMS (record-from-docs).

- Deferred items carried in writing with a trigger: EAD integration when customs can reach brokers; China customs in 2025; import items next.

Who I influenced: the Customs Platform engineering lead (agreed ownership of AES and ISF filing, subject to his business team, and preferred API-only integration) · Customs Tech's business owners (where each filing item lives) · FFA engineering (owns the operational items end to end) · the operations leads who had been doing EAD and portfiling by email.

TRADE-OFF I MADE

Options: (a) push every customs-adjacent item to Customs Platform, which is cleaner on paper; (b) build everything inside FFA, which is faster for us; or (c) split by qualification and operational ownership, with APIs at the seams. I chose (c). I rejected (a) because Customs had no plan to integrate with external brokers, so items would sit unowned. I rejected (b) because it would have duplicated licensed filing logic. It cost leaving ISF and AES unresolved through the first session and carrying EAD offline for now.

Failure / learning: The first session ended with two items, AES and ISF filing, still open, because the right business owner wasn't in the room. Lesson: get the business owner of every regulated item into session one, not just the engineering lead.

Earn Trust — two teams with misaligned goals left with one table both signed off on.

Invent & Simplify — one owner per row, with APIs at the seams instead of shared code.

Success & Scale Bring Broad Responsibility — cost-to-serve became a shared goal, not one team's metric.

Ownership — an un-owned overlap, picked up and closed.

PROBING — FOUR QUESTIONS HE WILL ASK

"Was this your document?"

I was on the FF App side of the split and worked the rows with Customs Platform engineering VERIFY. The attendee lists on both meeting notes don't include me, so confirm my exact role before telling this. If I wasn't in the work, drop R4 and use R2 instead. If he presses on authorship: "I'll be precise. The document and the meeting notes record the agreement, and my role was the FF App program side of it."

"Who disagreed?"

On AES and ISF filing, Customs engineering was willing to own them but needed their business team to decide where the item lived, and preferred API-only integration if it stayed in FFA. They were right that a licensed filing shouldn't be rebuilt outside customs, so we took the API route.

"How did you measure success?"

Every 2023 export item had one owner and one integration contract before build planning. The honest gap: the document records the agreement, not a later delivery metric, so I don't claim a downstream number.

"What would you do differently?"

Start with the import items in parallel. They followed later, and the same table would have taken one more session.

SAFE-SAY: "Fifteen work items on the 2023 plan, twelve export and three import. The agreement was written on November 3rd, 2022: the app owns every export item except ISF filing."

"What's your source?" "The ownership proposal and its two sets of meeting notes, October 26 and November 3, 2022."

20-SECOND VERSION

"Two tech orgs at Flexport were both building export customs, with fifteen work items for 2023 and no owners. I split it by principle: customs owns filing, the forwarding app owns moving the cargo. We worked one table row by row, with APIs at the seams. In two sessions, every export item had an owner except ISF, and both teams shared the cost-to-serve goal."

R5 · One trace ID through six systems — observability and incident response on a nine-team platform SV FAMILY — DIFFERENT ANGLE

Company: Flexport, Shipment Visibility (SV) launch and the 3PL ingestion test, 2021–22 · **My role:** TPM for the SV initiative across 6 systems and 9 teams · **Sources:** Shipment_Visibility_3PL_ingestion One-Pager p1–8; State_of_SV_E2E_Connectivity_Monitoring p1–8; Path_to_4PL p1–2; TPM status 11/19/21 p2, p10
Headline: blocker-bug MTTR −50%; troubleshooting had taken 2–4 hours just to find the system holding the bug (record-from-docs). **Not the wave/CTS story the HM heard**: this is the infrastructure layer underneath it.

60-SECOND PITCH

"At Flexport I was the TPM on Shipment Visibility: six systems, nine teams, close to a hundred people. In the bug bash I was reading application logs alongside the engineers, and I noticed there was no unique ID tying an event as it flowed from the downstream systems up to the customer screen. Every blocker took two to four hours just to find which system the bug lived in, before anyone could fix it. I recommended a design change: one traceable load-event ID end to end, plus a metric at every hop, events arriving divided by events sent, and events dropped divided by events planned. I wrote it up, got the engineering managers on Visibility, Ocean and Flexport OS to slot the stories, and paired it with Datadog monitors on every connectivity point and a list of kill switches before any production test. Time to fix a blocker bug fell by half. The lesson I wrote into our retrospective: monitoring should have come earlier in the lifecycle."

S — Situation. SV launched in December 2021 on new infrastructure integrated into the Flexport platform. **9 teams, close to 100 people** (Path to 4PL p1). Data travelled through **six or more system layers**, many too new to have robust monitoring. During testing, finding where a bug manifested took **2–4 hours** before an engineer could start fixing it (one-pager p1; Path to 4PL p2).

T — Task. As TPM I was responsible for delivering SV across the 6 systems and 9 teams and for running the bug bash across engineering and product. No engineer reported to me.

A — Action.

- **Read the logs myself.** In the bug bash I triaged in the application logs with the team and found the gap: no unique ID tying events from downstream to the user-facing interface.
- **Propose the design change with a metric.** A unique, traceable load-event ID through Visibility → Ocean → Flexport OS → Client App, and a load-event tracing metric at each hop: *events in originating system ÷ events in destination system* and *events dropped ÷ events planned*.
- **Get it slotted across teams.** Trace stories were on track on the Visibility and Ocean teams, and the design doc was completed for Flexport OS (one-pager p3).
- **Monitors before customers.** A Datadog monitor and dashboard for every connectivity point (tracking-request submission, forwarding stream blockage and P95 event age, gRPC subscriptions, Kinesis producer errors, API latency and availability) so we were customer-ready on 11/19 (E2E monitoring doc p1–8).
- **Blast-radius control for the next test.** For the 3PL ingestion test I listed the switches to turn off (for example the Ocean subscription kill switch and the trade-graph extract switches), restricted testing to named client and trade-lane combinations, and reused the holiday support framework for blocker and critical issues (one-pager p5–6).

R — Result.

- **Blocker-bug MTTR −50%** after the trace-ID design change (record-from-docs, the one-pager's own result line).
- Every SV connectivity point had a named monitor in production before the customer-ready date (record-from-docs). Performance tests completed and testing moved from staging to production (TPM status 11/19/21).

- The 3PL test plan made load-event tracing the success metric and excluded performance and cost-to-serve on purpose (one-pager p2).

Who I influenced: the engineering managers on Visibility, Ocean and Flexport OS (three backlogs carried a story none of them owned alone) · product (prioritize engineering-found issues; spot-check 10% of shipments) · the DRI and business leadership (keep the test to named clients and lanes).

TRADE-OFF I MADE

Options: (a) keep fixing bugs as they came, which is faster this week, or (b) spend sprint capacity on a cross-system trace ID and monitors that no customer would see. I chose (b). I rejected (a) because at 2–4 hours per bug just to locate it, the bug bash would never converge across six layers. It cost feature capacity on three teams during a delayed pilot.

Failure / learning: Our own retrospective says it: internal system monitoring came too late in the lifecycle, and test-effort estimates were too optimistic (Path to 4PL p2). On any new platform, I now put the trace ID and the monitors in the first sprint, not the bug bash.

Ownership — a TPM reading application logs and proposing an architecture change outside his remit.

Dive Deep — created the load-event tracing metric that made drops visible per hop.

Are Right, A Lot — brought engineering, product and three teams' perspectives into one design.

Bias for Action — the production test account decision (see New-since-this-morning, N4).

PROBING — FOUR QUESTIONS HE WILL ASK

"MTTR down 50% — measured how?"

On blocker bugs, comparing the troubleshooting time before and after the trace ID. That's the result line in my own one-pager. The before was 2–4 hours just to locate the system. I don't have a per-bug distribution in the doc, so I say "about half".

"Why was a TPM in the application logs?"

Because the bug bash was stalling and I was the only person looking across all six systems at once. Each engineer saw their own hop, and nobody saw the seam.

"Who pushed back?"

An engineering manager pointed out that the downstream system couldn't carry the ID yet. That was right, so it went in as a dependency with its own story and design doc rather than a hack.

"What would you build first at Amazon Infrastructure?"

The trace and the hop-level drop ratio before the first integration test, and monitors per connectivity point owned by the team that owns the hop. Then a kill-switch list per launch, reviewed at go/no-go.

SAFE-SAY: "Time to fix blocker bugs fell by about half after the trace-ID change. That's the result I recorded in my own one-pager at the time."

"What's your source?" "The SV 3PL-ingestion one-pager and the E2E connectivity-monitoring doc. The 'monitoring came late' lesson is in the 2021 Path-to-4PL review."

20-SECOND VERSION

"Six systems, nine teams, and no ID tied an event to its source, so each blocker took two to four hours to locate. I proposed one trace ID end to end with a drop ratio at every hop, got three teams to build it, and put a Datadog monitor on every connectivity point. Time to fix a blocker bug fell by half."

R6 · Two programs, one team, one ruler — the compressed go-live FREE

FF APP FAMILY — DIFFERENT ANGLE

Company: Flexport, 2022 · **My role:** program lead running my own program while a strategic go-live was pulled forward onto my shared engineers · **Source:** Apple v11 B3 p20–21 (dossier PM-4; 2022 onsite Senior TPM / Ambiguity doc)
Headline: go-live pulled forward a month; ~$200K a month per DC of savings at stake (record) → compressed program landed inside four weeks, mine on its original date; about $1M per DC protected (cumulative framing, record — quote the rate first).

60-SECOND PITCH

"At Flexport in 2022, leadership pulled a strategic go-live forward by a month and shared my engineering team across both programs. Two programs, one team, an immovable date, and about two hundred thousand dollars a month per DC of savings at stake. Arguing priority would have wasted the month. Instead I ran both as one system: one RAID log, one forty-five-minute daily forum, and one ruler for every issue. Is it a true blocker, a non-blocker, or a requirements gap pretending to be a blocker? Within a week most 'blockers' were reclassified. The true ones went to a two-pass war room, onsite then offshore with a written handoff, so work moved around the clock without anyone working twenty-four hours. Go/no-go read the blocker burn-down against an agreed floor, not the calendar. The compressed program landed inside four weeks, mine held its original date, and the business adopted on time."

S — Situation. Leadership pulled a strategic go-live forward one month and shared my engineers across both programs. About **$200K a month per DC** of savings was at stake (record). The metric in danger was on-time business adoption: a compressed landing the floor rejects is a miss that compounds.

T — Task. Land both programs without sacrificing the floor's adoption, with half a team, no new headcount and a date I didn't set.

A — Action.

- **One system for two programs.** One RAID log (issue, class, owner, date, program) and one 45-minute daily forum that ruled on both.
- **One ruler.** True blocker, non-blocker, or requirements gap in disguise. Most "blockers" were reclassified within about a week, so engineering stopped context-switching on non-bugs.
- **Two-pass war room.** Onsite pass, then offshore pass, with a written handoff: issues moved around the clock and nobody worked 24 hours.
- **The floor, not the calendar.** Go/no-go read the true-blocker burn-down against a floor agreed in advance.
- **Escalations as priced decisions.** "A costs this, B costs that", raised the same day, never as complaints.
- **Protect the floor.** A ten-step quick-reference guide for the DC users before cutover.

R — Result.

- **Both programs landed**: the compressed one inside four weeks, mine on its original date (record).
- The business **adopted on time**, protecting **about $1M per DC** of savings that would have slipped into the next year (record. This is the cumulative framing of the ~$200K/month rate, and I quote the rate first).
- Nothing public slipped. The cost was the team's calendar, and I said so.

Who I influenced: leadership (escalations framed as priced choices, so decisions came the same day) · the shared engineers (one ruler, one forum, so they stopped serving two masters) · the offshore team (written handoff instead of overlap hours) · the DC floor (the quick-reference guide).

TRADE-OFF I MADE

Options: (a) fight the pull-forward to protect my program, (b) let my program slip quietly, or (c) accept the decision and make the collision visible on one ruler. I chose (c). I rejected (a) because once the pull-forward was priced it was probably right, and (b) because a hidden slip is found in week two. It cost the team's calendar: two war-room passes a day for weeks. I guarded the 45-minute forum hard so the load didn't grow.

Failure / learning: The ten-step quick-reference guide went out the week before cutover, but floor questions had been arriving for weeks. Under compression, documentation dies first. The guide was cheap, and I made it late.

Bias for Action — the ruler and the forum were running inside week one.

Have Backbone; Disagree & Commit — I pushed for visibility, not against the decision, then committed fully.

Frugality — two programs landed on one shared team with no added headcount.

Strive to be Earth's Best Employer — round-the-clock velocity without anyone working round the clock.

PROBING — FOUR QUESTIONS HE WILL ASK

"$200K a month becomes $1M per DC — over what window?"

The monthly rate is the number I'd defend cold: priced savings per DC per month in the program doc. The million is the cumulative protection framing from the same document. Pressed, I quote the rate and the sub-four-week landing.

"Isn't 'blocker, non-blocker, gap' just relabeling while things burn?"

It's triage. The proof is the count: true blockers burned down to the agreed floor, and go-live read the floor.

"Did you fight losing the engineers?"

I pushed for the cost to be visible, not against the decision. Disagree-and-commit is only real if the disagreement was visible first.

"What did you do versus the team?"

I ran the system: the log, the forum, the ruler, the escalation framing, the guide. The engineers built and ran both war-room passes, and the floor adopted.

SAFE-SAY: "About two hundred thousand dollars a month per DC was at stake. The compressed program landed inside four weeks, and mine held its date."

"What's your source?" "The 2022 program doc and my Senior TPM write-up of that quarter."

20-SECOND VERSION

"Leadership pulled a go-live forward a month onto my shared engineers. I didn't argue priority. I put both programs on one RAID log, one 45-minute forum and one blocker ruler, with a two-pass onsite-offshore war room. Both landed: the compressed one inside four weeks, mine on its date."

R7 · The LLM triage agent — $25 to $0.40 an incident, on an eval set and a citation rule

FREE

Context: independent build (not an employer program), recent · **My role:** everything: Python, FastAPI, LLM tool use, SQLite, the eval set, the cost engineering · **Source:** Apple v11 I1 p41–43; resume (LLM triage agent) **Headline:** cost per triaged incident $25 → $0.40 (−98%) with accuracy held on the eval set; citation coverage 100% (record).

60-SECOND PITCH

"I built an LLM triage agent for operational incident queues with my own hands. It reads the ticket, classifies it and proposes the first action with citations. The naive version worked, but it cost twenty-five dollars an incident. That's a demo, not an operations tool. I ran it like a program with a budget line. First, an eval set before the agent: graded historical incidents with known-correct outcomes, and every prompt, model or routing change had to keep the past passing. Second, a cost bar of under a dollar an incident, reviewed on every change. Third, one rule: no citation, no proposal. Every proposed action cites the ticket evidence and the runbook line. Then I routed the easy classes to small models and saved the expensive calls for the hard tail. Cost fell from twenty-five dollars to forty cents, a 98% cut, with accuracy held and citations at 100%."

S — Situation. Operational incident queues need a first triage: read, classify, propose the first action. A straightforward LLM agent did it at about **$25 per incident** (record), which is too expensive and too untraceable to put in front of an operator.

T — Task. My own build, end to end. Targets: triage accuracy held on known cases, citation coverage on every proposal, and cost per incident under $1.

A — Action.

- **Eval set before the agent.** Historical incidents with the triage outcome the organization already agreed was right: graded, versioned and growing. The contract was "change anything, but the past must keep passing".
- **Cost as a budget line.** The bar was under $1 per incident, reviewed on every change like a program budget.
- **No citation, no proposal.** Every proposed action binds to ticket evidence and a runbook line, so a human can check it in seconds.
- **Routing, not heroics.** A cheap classifier routes easy classes to small models, and expensive calls are reserved for the hard tail. The eval set proved the cheap path held accuracy before traffic moved.
- **One pane.** Triage accuracy on the eval set, citation coverage and cost per incident, the same three numbers every review.

R — Result.

- Cost per triaged incident **$25 → $0.40, −98%**, with accuracy held on the eval set (record).
- **Citation coverage 100%**: the agent never proposed without evidence (record).
- Beat the sub-$1 bar. The eval-set discipline is what I'd bring to any AI-in-operations program.

Who I influenced: nobody reported to me, and this is a build story, not an org story. The influence is downstream: the eval set and citation rule are the contract I'd hand an operations team before they trust an agent's proposal.

TRADE-OFF I MADE

Options: (a) one strong model for everything, which is simplest and most accurate per call, or (b) tiered routing with an eval gate. I chose (b). I rejected (a) because at $25 an incident nobody would run it at volume. It cost building and maintaining the eval set first, which was slower to start and a standing upkeep cost.

Failure / learning: I didn't version the eval set on day one. Early prompt changes were judged on a set that was still growing, so the first accuracy claims leaned on a moving baseline. Now I freeze version one first.

Frugality — −98% cost per incident, engineered like a budget line, quality held.

Invent & Simplify — routing plus one rule (no citation, no proposal) instead of a bigger model.

Learn & Be Curious — learned LLM engineering by building, not reading.

Insist on the Highest Standards — the past must keep passing before a change touches the present.

PROBING — FOUR QUESTIONS HE WILL ASK

"How did cost drop 98% without dropping quality?"

Mostly routing. Most incidents are easy, and a small model classifies them fine. The eval set made it safe, because I proved the cheap path on the easy classes before routing traffic there.

"Where would this break at Amazon's scale?"

The volume economics and the data-sensitivity review. Eval-set maintenance becomes a program of its own, and incident data carries customer content that needs an access review. That's my security-review scar (F2), and I'd schedule it at kickoff.

"Isn't this a side project?"

It's an independent build, and I say so. I tell it because it shows the same mechanism as my programs: a baseline, a gate, a budget line and a single pane, applied to AI.

"What's the eval set, concretely?"

Graded historical incidents with the agreed-correct triage, versioned, and every change runs against it first. It's the contract.

SAFE-SAY: "Twenty-five dollars to forty cents per triaged incident, about 98% lower, on my own independent build, with accuracy held on a graded eval set."

"What's your source?" "My own build and its eval and cost logs. It's on my resume as the LLM triage agent."

20-SECOND VERSION

"I built an LLM triage agent myself. The naive version cost twenty-five dollars an incident. With an eval set first, a no-citation-no-proposal rule, and small models routed to the easy classes, it runs at forty cents, 98% lower, with accuracy held and 100% citations."

R8 · One log, one grammar — turning an 836-row issues log into the go-live gate

P1 — DIFFERENT ANGLE (THE MECHANISM, NOT THE LAUNCH)

Company: Sephora, UDC dual-inventory conversion, 2024–25 · **My role:** program lead; I owned the instruments: the consolidated issues log and the coupled build plan · **Sources:** Apple v11 C2 p23–25; UDC_Consolidated_Issues_Log (pandas, 836 rows); NUMBERS-RECON canon #44–46, #52
Headline: 836 rows over 2024-03-17 → 2025-10-01; 714 closed = 85.4%; post-cutover spike of 271 rows (June 2025) burned down 65 → 21 → 14 → 4; zero customer-facing incidents (record). **Tell only if P1 has not already been told in full**. The angle is "a mechanism I built", not "the launch".

60-SECOND PITCH

"On the UDC conversion at Sephora there were about thirty build tracks and sixty-plus stakeholders. Status by opinion would have drowned us. So I built one instrument: one consolidated issues log with one grammar. Every row had a status, priority, operation, functional area, owner and date, and there were no side lists. The weekly review read blocker and critical rows first, and every red had a name and a date. Then I made the log the go-live gate itself: both go-lives read burn-down curves, not confidence. Over 836 rows, only about one in seven was actually a bug. The rest were requirements, action items, questions and parked decisions, so the log was the program's work-triage system, not a defect list. After the combined go-live the log spiked to 271 rows in a month and burned down 65, 21, 14, 4. That's a building healing, not a design problem. 85% closed, zero customer-facing incidents."

S — Situation. A live Utah fulfillment center was converting from two systemic warehouses into one omni facility. That meant **about thirty build tracks**, **60-plus stakeholders** and about a dozen systems (record). The at-risk read was go-live readiness: defect burn-down, track completion and cutover preparedness.

T — Task. Build the instruments so a surprise surfaced in a Monday review, not in cutover week, and make readiness a count rather than a feeling.

A — Action.

- **One log, one grammar.** Status ladder, priority scale, operation, functional area, owner, date. One row per issue. Dedupe was part of the rhythm.
- **Read the dangerous half first.** Blocker 164 + Critical 277 = 441 rows = 52.8% of the book (record). Those rows led every weekly review.
- **A parking-lot status that means "decided not now".** Written down with an owner and never silently deleted, so deferral was a decision, not drift.
- **The log is the gate.** Both go-lives (PO creation January 2025, combined May 2025) read burn-down curves against the bar (record).
- **Owner split as a resourcing signal.** IT/WMS 342, Ops 111, Locus-side 86. When an owner's queue grew two weeks running, I opened a resourcing conversation with the evidence attached.

R — Result.

- **836 rows, 714 closed = 85.4%**, with the open tail visible and owned (record, recomputed from the log).
- Type mix: **bugs 115 (~14%)**, requirements 159, action items 117, questions 23. It worked as a triage system, not a defect fire (record).
- June 2025 spike of **271 → 65 → 21 → 14 → 4**: a healthy cutover decay (record).
- **Zero customer-facing incidents** at both go-lives (record).

Who I influenced: about fifteen functions and the vendors (Blue Yonder WMS, Locus) to write into one log in one grammar · the IT/WMS lead (queue-growth evidence became a resourcing ask) · the steering committee (read the

curve, not the RAG color).

TRADE-OFF I MADE

Options: (a) let each track keep its own tracker, which is lighter for them, or (b) one log with enforced grammar and a weekly dedupe. I chose (b). I rejected (a) because two trackers mean two truths and blockers hide in the gap. It cost overhead on every track lead each week, and some friction from vendors used to their own tools.

Failure / learning: I'd put the monthly-shape view (ramp, spike, burn-down) on the wall from the first hundred rows. The counts were always there, but the arc was a manual read well into the program.

Insist on the Highest Standards — readiness came from counted rows against a zero-incident bar, twice.

Think Big — the log was redesigned from defect list into the program's operating system.

Dive Deep — only 14% bugs, and the owner split told the integration story.

Ownership — the instruments were mine; nobody else owned "the truth on Monday".

PROBING — FOUR QUESTIONS HE WILL ASK

"836 issues — wasn't that a fire?"

No, and the log itself shows it: 115 bugs, about one in seven. The rest were requirements, action items, questions and parked decisions. Every "should we?" got a row instead of a hallway conversation.

"How do you tell healing from a design problem?"

The shape. A spike with fast decay, 271 then 65, 21, 14, 4, is a healthy cutover. A spike that plateaus is a design problem.

"What did the vendors think of your grammar?"

Locus had its own tracker. Their rows came into the log with the same fields, 86 of 836, about a tenth, which made the robotics workstream's noise measurable inside a WMS conversion.

"Where did the plan slip?"

Locus retail testing slipped three weeks. It was re-slotted openly in the plan's revision sheet, not absorbed into weekends. That's failure F4.

SAFE-SAY: "836 rows from March 2024 to October 2025, 85.4% closed, about one in seven a bug, and zero customer-facing incidents at both go-lives."

"What's your source?" "The UDC consolidated issues log itself. I recomputed the counts from the export."

20-SECOND VERSION

"On a thirty-track warehouse conversion I built one issues log with one grammar and made it the go-live gate. 836 rows, only one in seven a bug, 85% closed. The post-cutover spike of 271 burned down 65, 21, 14, 4, with zero customer-facing incidents."

R9 · The QA lead who wrote the gate — growing owners in a 20-person team with no mandate

ALSO WITH JOSYULA — DIFFERENT ANGLE

Company: Sephora, JDA years, soon after I joined · **My role:** VP-sponsored lead for converting a 20-person product, engineering and QA team to agile delivery, with no line authority · **Source:** Apple v11 D3 p31–32; resume; master S15

Headline: six-week hybrid → four-week cadence in six weeks; critical defects −50%; copied by neighbouring teams (record). **Angle for Rocky: Hire & Develop / Earth's Best Employer.** The people I grew into owners. Josyula's telling is the influence-without-authority angle.

60-SECOND PITCH

"When I joined Sephora, the VP of supply chain asked me to move a twenty-person product, engineering and QA team from a six-week hybrid model to a four-week agile cadence. I had sponsorship but no mandate, and the team assumed four weeks just meant more work in less time. QA was the most afraid, because they expected their testing window to be crushed. Instead of designing around QA, I made the QA lead the author of the definition of done, with entry criteria. Their biggest fear became the design, and the QA lead owned the gate in front of the whole team. I ran one-on-ones across all three functions first, named an influencer in each, and published one chart every cycle showing what moved and what didn't. In six weeks we were on a four-week cadence, critical defects fell by half, and neighbouring teams copied it without being asked."

S — Situation. The team kept missing high-story-point work, and the business saw no incremental value. Twenty people across product, engineering and QA. VP sponsorship, no mandate (record).

T — Task. Convert to a predictable four-week cadence, and make the team own it so it would survive without me.

A — Action.

- **People discovery first.** One-on-ones across all three functions, mapping objections, motivations and who actually influenced whom. Then a named influencer in each function with specific objectives.
- **Give the hardest job to the most skeptical person.** The QA lead designed the definition of done with entry criteria. I coached by asking "what would make this safe for QA?" and let them write the answer.
- **Their ceremonies, my evidence.** The team authored the ceremony details. I built the Jira hygiene underneath: a business-benefit index so high-value stories slotted first, and a review-status gate before QA.
- **Make the work visible.** One weekly chart of what moved to QA on time, what moved and what didn't, failures included.
- **A written workflow guide** from grooming to done, so every status was captured the same way whoever touched it.

R — Result.

- **Hybrid → four-week cadence in six weeks** (record).
- **Critical defects −50%** (record, resume).
- Adoption held without a mandate, and **neighbouring teams copied the cadence** on their own (record).
- The QA lead's definition of done became the team's gate. It was later the QA gate my own in-transit fix had to pass (F3).

Who I influenced: the VP (air cover, not orders) · the QA lead (from blocker to author of the gate) · one named influencer each in product and engineering · neighbouring team leads (copied a result, not a name).

TRADE-OFF I MADE

Options: (a) draft the ceremonies and the definition of done myself, which is faster, or (b) let the team author them. I chose (b). I rejected (a) because a cadence imposed by me would die when I left the room. It cost control and time: drafts I could have written in a day took weeks of their sessions, and their version wasn't the one I'd have written.

Failure / learning: I should have gone to QA first. Their fear was a design requirement disguised as a complaint, and starting with the most skeptical function would have shortened the six weeks.

Hire & Develop the Best — I coached the QA lead into owning the team's quality gate.

Strive to be Earth's Best Employer — I advocated for QA's working window inside the design.

Think Big — drove adoption of a vision without a mandate, and it was copied by neighbours.

Earn Trust — published failures as visibly as wins.

PROBING — FOUR QUESTIONS HE WILL ASK

"You had no direct reports. Is this really developing people?"

Honestly, it's developing owners, not a direct report's career. I've led through programs, not org charts, and I say that up front. What transfers is making people who don't report to me own a standard.

"How did you coach the QA lead specifically?"

I gave them the blank page, not a draft. I asked them to define what "ready for QA" means in testable entry criteria, reviewed it with them before the team saw it, and let them present it.

"How do you know it was the cadence and not luck?"

The weekly chart before and after, and the critical-defect count halving. The strongest evidence is that other teams copied it without being asked.

"Raising the bar: who would you hire for this team now?"

Someone who writes the gate rather than waiting for one. The QA lead was that person once given the pen.

SAFE-SAY: "Twenty people, hybrid to a four-week cadence in six weeks, critical defects down by half. I had no direct-report org, and I say so."

"What's your source?" "My resume and the team's own cycle charts."

20-SECOND VERSION

"Twenty people, sponsorship but no mandate. I gave the most skeptical person, the QA lead, the pen on the definition of done. Their fear became the gate. Four-week cadence in six weeks, critical defects halved, and neighbouring teams copied it."

4 · Failures — Bar Raisers always ask (F1–F4) NEW

Rules: never open with these; own the miss in the first sentence; root cause in my own process; end on the standing control that outlived the program. G2 (the RF-device script) stays with P7 in the program packet. Choose by audience: **F1 is the strongest** (a customer-facing miss). Use F3 only if Ryan did not hear S4 yesterday.

F1 · The UAT that crashed — the performance criteria I never wrote FREE FAILURE

Company: Elementum, track-and-trace application · **Source:** Apple v11 G1 p43–44 (2022 onsite Failure doc) · **Numbers:** 5 key users × 10 E2E scenarios; crash 2 days before sign-off (record).

60-SECOND PITCH

"I lost a UAT sign-off at Elementum, and the failure was mine. Two days before sign-off, five key users had passed all ten end-to-end scenarios, and I was confident. Then the sponsor escalated: the system was buggy, and users couldn't finish a single flow. My first move was wrong. I re-ran the scenarios, they passed, and I sent screenshots saying it worked. I was defending instead of digging. The real cause: the sponsor had opened access to all their users, and concurrent logins crashed a system my UAT criteria never load-tested. I took it to the sponsor as my miss while the team simulated the concurrency and fixed it around the clock. The sign-off date passed. Since then every gate I run carries a load number. At Flexport, the launch gate required 500 containers in 20 minutes at 99.6% connectivity before any customer saw the product."

S — Situation. Track-and-trace UAT, sign-off two days out. **5 key users had completed all 10 E2E scenarios** (record). The sponsor escalated: buggy, flows unfinishable. The launch date and user confidence were at stake.

T — Task. Recover the launch and the sponsor's trust, and find out why my UAT said "pass" while the customer said "broken".

A — Action. (1) My first move was the failure: I re-ran the scenarios and emailed screenshots. (2) Then first principles: the noise said "buggy". The cause was concurrency, because the sponsor had opened access to all their users. (3) I took the miss to the sponsor as mine: my criteria covered explicit needs and ignored the implicit one, performance. (4) My team simulated the concurrency and shipped the fix round the clock. (5) I rewrote the UAT contract: numeric entrance and exit criteria on every critical scenario (uptime, page load, concurrent load), tested at the customer's shape.

R — Result. The sign-off date passed and confidence had to be rebuilt (record: missed). **The standing control:** every gate since carries a load line. The Flexport SV gate's performance proof was **500 containers in 20 minutes at 99.6% connectivity** before any customer testing (record-from-docs, TPM status 11/19/21), and the JDA UAT entrance criteria held.

TRADE-OFF I MADE (AND THE ONE I'D MAKE NOW)

Under deadline I traded depth of testing (five users, scenario coverage) for speed to sign-off, and didn't price the load risk. Now I'd trade two days of schedule for a concurrency test at the customer's user count every time.

Level 1, "What exactly did you do wrong?" I wrote criteria that tested what users asked for and not how they'd use it, and when it broke I defended with screenshots before reproducing their experience. The screenshots email is the moment I'd take back.

Level 2, "Wasn't the sponsor at fault for opening access?" No. Users use the system the way their day demands. It was an implicit requirement I failed to surface, so my criteria owned the gap.

Level 3, "How do you know the lesson stuck?" A number in a later program: the SV performance proof of 500 containers in 20 minutes at 99.6%, run before customers. And my first hour on any escalation now goes to reproducing the customer's shape, not proving our innocence.

Are Right, A Lot (AIQB Q5 — an error in judgment) · **Earn Trust** — owned it to the sponsor by name · **Insist on the Highest Standards** — the load line on every gate · **Customer Obsession** — the customer's experience was the truth.

SAFE-SAY: "Five users, ten scenarios, all passed, and it crashed under concurrent logins two days before sign-off. The sign-off date passed. That one was mine."

"What's your source?" "My own failure write-up from that program."

20-SECOND VERSION

"I lost a UAT sign-off because I tested the scenarios and never the load: concurrent logins crashed it two days out. Worse, I defended with screenshots first. Now every gate I run has a load number, like the 500 containers in 20 minutes at 99.6% before Flexport customers saw SV."

F2 · The security review I discovered instead of planned FAILURE

ALSO WITH JOSYULA — DIFFERENT ANGLE

Company: Flexport, the Snowflake pipeline that automated the weekly business review · **Source:** Apple v11 G3 p45–46; BRD SV Zendesk Data in Snowflake · **Numbers:** access review under-planned by two weeks (record); cut over on a ≤1% parallel-run gate (record). The HM heard the pipeline story, so I tell only the planning miss here.

60-SECOND PITCH

"A smaller failure with a long shadow. At Flexport I was replacing my manual four-system weekly review with a Snowflake pipeline, on ticket data that carried customer content. I had planned the extracts, the loads, metric parity and the parallel-run cutover gate. I had not planned the data-access security review. It surfaced mid-build and cost two weeks. I owned it upward the same day: the slip was my scoping, not engineering, because my BRD priced the build and never the review. I re-sequenced instead of stalling. The parallel run kept the Monday review alive on the manual path, so the program absorbed the two weeks without skipping a review. The control that outlived it: security lead time is planned, not discovered. It's a standing line on every data program since, with one kickoff question: whose approval does this data need, and how long does it take?"

S — Situation. Automating the weekly business review onto a Snowflake pipeline, on Zendesk ticket data carrying customer content. The plan covered build and parity, but not the access review.

T — Task. Land the pipeline without breaking the weekly review cadence, and fix the planning gap, not just this instance of it.

A — Action. (1) Owned it upward immediately as a scoping miss. (2) Root cause: I'd treated the access review as a formality, not a workstream with lead time, and customer content makes it a gate. (3) Re-sequenced: the manual path stayed live in parallel. (4) Closed the review properly: data classes named, who could read what, cross-reference table documented. (5) Added the kickoff checklist line.

R — Result. **Two weeks lost** on the review (record), **zero skipped review weeks**, and cutover on the manual-versus-pipeline **≤1% parity gate** (record). **Standing control:** security and access lead time is a named line with an owner, priced in the BRD next to build cost. The I1 agent had access boundaries designed in from day one.

TRADE-OFF I MADE

When it surfaced: (a) push the review through fast and cut over on time, or (b) keep the manual path alive two more weeks and close the review properly. I chose (b). It cost me another two weeks of hand-stitching the Monday review myself.

Level 1, "Two weeks is small. Why is it a failure?" Because it was structural, not situational. Every data program with customer content has an access review, so without a fix I'd hit the same wall on every one.

Level 2, "Did the parallel run save you by design?" By design and by luck. I built it as a correctness gate, and it happened to double as a schedule buffer. Now I plan for that: the old path stays live until the new one clears both correctness and access.

Level 3, "What's different at Amazon?" Amazon's data-sensitivity bar makes the access review the longest pole on most data programs. I'd schedule it at kickoff, with the owner and the lead time on the plan.

Ownership (AIQB Q2 — a commitment at risk, communicated) · **Earn Trust** — owned upward the same day · **Are Right, A Lot** — a systemic cause behind a small miss.

> **SAFE-SAY:** "The access review cost two weeks because I hadn't planned it. The Monday review never skipped, because the manual path stayed live."
>
> **"What's your source?"** "My program notes and the BRD for the Snowflake pipeline."

20-SECOND VERSION

"I planned the pipeline and forgot the data-access review. It cost two weeks. I owned it the same day and kept the manual review alive in parallel, so nothing skipped. Since then, security lead time is a kickoff line on every data program."

F3 · The in-transit promise rule I got wrong FAILURE

CHECK: RYAN'S ROUTER HAD S4 AS HIS FAILURE ANSWER

Company: Sephora, S/4HANA ↔ ISM availability model · **Sources:** GFiber v8 S4 p66–67; master S4; NUMBERS-RECON canon #56 · **Numbers:** lane σ ≈ 0.3 day → ~0.49-day promise window ≈ 95% coverage; oversell increase zero; kill switch never fired (all **user anchor** — Apple has the shape only). No dollar figure for the conversion I lost, and I won't invent one.

60-SECOND PITCH

"The first availability ruleset I wrote for Sephora's new inventory model barred every channel from promising units in transit. It felt safe. Within weeks of go-live, the Aging In-Transit and replenishment reports showed dotcom starved of fast movers that were reliably one day from the building, and we were losing conversion on those SKUs. The rule was mine. I built the evidence pack, said so in the first line, and reframed it as promise math rather than 'make in-transit count'. The new rule: promise in-transit only where the lane's ETA falls inside a per-lane window from receiving history, with an on-time floor and the aging report as a kill switch. I took it back through the same three signatories and released it on the normal monthly cadence as a signed change, not a side configuration. Sellable units came back on qualifying lanes, oversell didn't rise, and the kill switch never fired. The lesson: both sides of a rule get a number before it ships."

S — Situation. My v1 ISM ruleset excluded in-transit from every channel's availability. The Aging In-Transit and replenishment reports showed fast movers reliably one day out, with dotcom starved and conversion lost on those SKUs (user anchor; Apple has the shape).

T — Task. Correct my own policy without weakening oversell protection, through the same signatories: Supply Chain Strategy, Inventory Management and Inventory Accounting.

A — Action. (1) Evidence pack first, with "the v1 rule was mine" in the first line. (2) Rule redesign: promisable in-transit = units whose ETA is inside the lane's window *and* whose lane meets an on-time floor. The window came from receiving history: lane σ about 0.3 day gives a window of about half a day at roughly 95% coverage (user anchor). (3) Rejected both blanket options (all in-transit, or none) in writing. (4) Back-tested against receiving history, then added four regression cells to the node × channel test matrix, passing the QA definition-of-done gate. (5) Released as a signed FSD change on the normal cadence, with the aging report armed as a kill switch.

R — Result. Sellable units restored on qualifying lanes, **oversell increase zero**, **kill switch never fired** (user anchor). **Standing control:** "both sides of a rule get a number before it ships". Every later gate was set from data before the event: the Locus 98% pick bar and the 0.5% migration tolerance.

TRADE-OFF I MADE

In v1 I traded sales for safety, zero in-transit promising, without pricing the sales side. In v2: (a) promise all in-transit, (b) keep none, or (c) promise per lane inside a window, with a floor and a kill switch. I chose (c). It cost a slower fix (signed change, regression, monthly release) instead of a quick configuration flip.

Level 1, "What exactly was your mistake?" I wrote a rule with a number on one side only: oversell risk, priced at zero tolerance. I never priced the undersell. The rule was mine, not the configurators'.

Level 2, "How much did it cost?" The record doesn't hold a dollar figure for the conversion lost while I was wrong, and I won't make one up. What I can give you is the shape: fast movers one day out, starved for weeks.

Level 3, "Why not just flip a config?" Because a side configuration would have created a second, unsigned truth about "available", the exact vocabulary bug the program existed to kill. Same signatories, same cadence.

Are Right, A Lot (AIQB Q4 — a bad decision) · **Success & Scale** (Q2 — changed a decision because of unintended consequences for customers) · **Earn Trust** — owned through the same signatories · **Insist on the Highest Standards** — signed change, not a side fix.

SAFE-SAY: "My first rule barred in-transit everywhere and starved dotcom. I fixed it through the same three signatories with a per-lane window, oversell didn't rise, and the kill switch never fired."

"What's your source?" "The Aging In-Transit and replenishment reports and the signed FSD change. The lane numbers are my program anchors."

20-SECOND VERSION

"I wrote an availability rule that never promised in-transit. It felt safe and starved dotcom of fast movers one day out. I owned it, rebuilt it as a per-lane window with a floor and a kill switch, and took it through the same signatories. Oversell stayed flat. Lesson: both sides of a rule get a number."

F4 · Locus retail testing slipped three weeks — a vendor dependency I carried as a task, not a risk FAILURE P1/P3 — TOLD ONLY AS A FAILURE

Company: Sephora UDC, Locus robotics retail track (RTL) · **Sources:** Apple v11 C2 p24–25 and C3 p26; UDC_LOCUS_Retail_Track status (YELLOW; UAT window 2/28–3/15); prior Rocky v2 packet (escalation and re-slot facts) · **Numbers:** RTL testing slipped 3 weeks (record); Locus-side rows 86 of 836 = 10.3% (record); root cause of the vendor slip itself VERIFY.

60-SECOND PITCH

"On the UDC conversion, Locus robotics' retail-track testing slipped three weeks. The slip itself was a vendor-and-integration dependency, but the failure I own is how I carried it. It sat on my plan as a task with a date, not on the risk register with a trigger and an owner. So when it moved, I learned it from the status report, not from a tripwire I had set. What I did right after that: I escalated the week it surfaced, re-slotted the end-to-end chain openly in the plan's revision sheet, and refused to compress UAT to absorb it. The go/no-go bar, zero open blocker and critical issues, was set before the pressure and held. The combined go-live landed in May 2025, with zero customer-facing incidents. The control I run now: every vendor dependency on the critical path goes on the risk register with a trigger date and a named owner on both sides."

S — Situation. Locus design and build ran July–December 2024, RTL end-to-end completion was due January 31, and physical testing ran February–March 2025 (record, Apple C3). The retail-track status went YELLOW with integration items still open: the API approach (sockets versus REST), the retail customer-owner configuration, and tote-type mapping (record-from-docs, Locus Retail Track). RTL testing slipped three weeks (record).

T — Task. Protect the combined go-live and its quality bar without hiding the slip or squeezing UAT.

A — Action. (1) Escalated the week it surfaced, with the re-slotted dates attached. (2) Re-slotted the E2E chain openly in the plan's revision sheet, never silently absorbed into weekends. (3) Held UAT at full length rather than compressing it to buy back the three weeks. (4) Kept the go/no-go bar as set: blocker and critical open = 0. (5) Moved vendor dependencies from the task list onto the risk register, each with a trigger date and owners on both sides.

R — Result. The combined go-live **landed in May 2025** with **zero customer-facing incidents** (record). Locus-side issues stayed at **86 of 836 rows, about a tenth** of the program's log (record). **Standing control:** vendor critical-path items live on the risk register with a trigger and an owner, and a slipped vendor doesn't get to lower the bar.

TRADE-OFF I MADE

After the slip: (a) compress UAT and keep every original date, or (b) re-slot openly and hold UAT length. I chose (b). It cost three weeks of float on the chain and an uncomfortable steering update.

Level 1, "Whose fault was the slip?" The slip came from the vendor and integration track. I don't have a single root cause in my record, so I won't make one up VERIFY. My failure was carrying it as a task. Nobody had a trigger to pull earlier.

Level 2, "How would a risk entry have changed anything?" A trigger date ahead of the January 31 E2E milestone, with Locus and our WMS lead named, would have surfaced it earlier, while the float was cheaper.

Level 3, "What happened after?" The robotics gate still did its job: week two measured 94% against a 98% bar, about 30% of volume was throttled to manual carts, and full automation came in week three (record, P3). Gates set before the event held.

Ownership (AIQB Q2 — a commitment at risk, communicated) · **Insist on the Highest Standards** (Q4 — standards over delivery: UAT not compressed) · **Deliver Results** (Q6 — something didn't complete on time).

SAFE-SAY: "Locus retail testing slipped three weeks. I re-slotted openly, didn't compress UAT, and the combined go-live landed in May 2025 with zero customer-facing incidents."

"What's your source?" "The UDC build plan's revision sheet and the Locus retail-track status."

20-SECOND VERSION

"Locus retail testing slipped three weeks. My failure was carrying a vendor dependency as a task, not a risk with a trigger. I escalated that week, re-slotted openly and held UAT, and the May go-live landed clean. Now every critical-path vendor item has a trigger and an owner on the register."

5 · STAR answer bank — all 16 Leadership Principles, two real AIQB questions each NEW

Each card: the AIQB question quoted (cited "AIQB: <LP> Q#"), a spoken opener, a ~90-second S/T/A/R, the number to land, what NOT to say, the source story, and the likely three-level follow-up with its answer. None of these 32 question numbers were the router questions used with the hiring manager yesterday (she got CO Q8, Own Q3, I&S Q1, ARL Q1, L&BC Q4, HDB Q9, HSS Q4, TB Q2, BfA Q1, Frug Q1, ET Q2, DD Q3, Backbone Q1, DR Q3, EBE Q3, BR Q5). **32 answers from 17 distinct stories.**

The 16-row matrix — LP · primary story · backup story · the number

LP	PRIMARY (AIQB Q#)	BACKUP (AIQB Q#)	NUMBER TO LAND (BASIS)
Customer Obsession	R1 routing guide (Q6)	F1 UAT crash (Q1)	DQ ~40% reported / ~55% true → 99% (record)
Ownership	R5 trace ID (Q1)	F2 security review (Q2)	Blocker MTTR −50% (record-from-docs)
Invent & Simplify	R2 carrier bid (Q5)	R7 triage agent (Q2)	6 weeks → ~3 (user anchor)
Are Right, A Lot	R5 trace ID (Q7)	F3 in-transit rule (Q4)	2–4 hrs to locate a bug → MTTR −50% (record-from-docs)
Learn & Be Curious	R7 triage agent (Q2)	P3 Locus robotics (Q1)	$25 → $0.40 per incident (record)
Hire & Develop	R9 QA lead (Q4)	R3 site super-users (Q3)	4-week cadence in 6 weeks; critical defects −50% (record)
Highest Standards	R8 issues log (Q6)	P3 98% robot gate (Q1)	836 rows, 85.4% closed, 0 customer incidents (record)
Think Big	R1 Austria → 25 locations (Q1)	R9 cadence copied (Q4)	~$8M across 25 European locations (record)
Bias for Action	R6 two programs, one ruler (Q8)	P6 Cyber Monday (Q4)	~$200K/month per DC at stake (record)
Frugality	R7 cost per incident (Q4)	R3 snapshot-diffing (Q3)	−98% per incident (record)
Earn Trust	R4 Customs split (Q5)	P7 RF devices (Q4)	15 work items, one owner each (record-from-docs)
Dive Deep	R5 tracing metric (Q6)	R1 true DQ metric (Q5)	events-in ÷ events-out per hop (record-from-docs)
Backbone	R6 disagree & commit (Q3)	R2 "not an optimizer" (Q2)	landed inside 4 weeks (record)
Deliver Results	P1 UDC staged go-live (Q1)	P2 JDA mocks (Q2)	+15% processing efficiency; 7.0% → 0.3% (record)
Earth's Best Employer	R6 two-pass war room (Q5)	R9 advocated for QA (Q2)	45-minute forum; nobody on 24 hrs (record)
Broad Responsibility	R4 shared CtS goals (Q1)	F3 changed my rule (Q2)	oversell +0, kill switch never fired (user anchor)

Customer Obsession

AIQB: CUSTOMER OBSESSION Q6 **"Tell me about a time when a customer came to you for something that wouldn't actually address their need. How did you approach the situation? What was the result?"**

Opener: "The client asked me to hold carriers to their on-time SLA. What they needed was to be able to measure on-time at all."

S At Elementum, a client's Austria transportation team believed carriers were late. The report showed 40% data quality, and they spent 8–10 hours a week reconciling dates by hand. **T** They asked for carrier enforcement. I owned the analysis and the rules design. **A** I pulled three years of shipment history in SQL and showed the sponsor that true quality was about 55%. Nobody could prove a carrier was late on that data. I proposed the thing they hadn't asked for: fix measurement first, with a 99% data gate before the routing engine scaled, and a savings method co-written with Finance. **R** Quality reached 99% within a quarter. The Austria pilot measured about 97% on-time accuracy and about $500K a year, and the 25-location rollout closed at about $8M validated. Then carrier performance could be enforced on real numbers.

Land: 55% true vs 40% reported → 99%. **Don't say:** "the customer was wrong"; any lane count; "$1M". **Drawn from:** R1.

3-level follow-up. L1 "How did you tell them no?" → I didn't say no. I showed their own three years of data and asked what an SLA is worth at 55%. L2 "What if they'd insisted?" → I'd have enforced on the cleanest lanes only and published the quality caveat next to every penalty. L3 "What did the customer say after?" → They adopted the method across 25 locations, and Finance reused it.

AIQB: CUSTOMER OBSESSION Q1 **"Describe a difficult interaction you had with a customer. How did you deal with it? What was the outcome? How would you handle it differently?"**

Opener: "The most difficult one was my own fault: a sponsor told me the system was broken, and I sent screenshots saying it worked."

S Elementum track-and-trace UAT, sign-off two days out. Five key users had passed all ten scenarios. **T** The sponsor escalated: users couldn't finish a flow. I had to recover the launch and the relationship. **A** My first move was wrong: I re-ran the tests and sent screenshots. Then I reproduced their real shape. They had opened access to all users, and concurrent logins crashed the system. I called the sponsor, said the criteria were mine and had missed load, and my team fixed it round the clock. **R** The sign-off date passed. Trust came back because I owned the root cause. Every gate since carries a load number, like 500 containers in 20 minutes at 99.6% at Flexport.

Land: 5 users × 10 scenarios; crash 2 days out. **Don't say:** "the sponsor opened access too early" as blame. **Drawn from:** F1.

3-level follow-up. L1 "What would you handle differently?" → First hour of any escalation: reproduce their experience, not prove our innocence. L2 "Why did you defend first?" → I mistook my test coverage for the truth. L3 "How did you rebuild trust?" → I named the root cause as mine and changed the criteria so this class of miss can't recur.

Ownership

AIQB: OWNERSHIP Q1 **"Tell me about a time when you took on something significant outside your area of responsibility. Why was it important? What was the outcome?"**

Opener: "As the TPM, my job was the plan. I ended up proposing an architecture change, because nobody was looking at the seam between six systems."

S Flexport's Shipment Visibility: 6 systems, 9 teams, close to 100 people, on new infrastructure. **T** I owned delivery and the bug bash, not the design. **A** I read the application logs alongside the engineers and found that no unique ID tied an event from the downstream systems to the customer screen. Finding which system held a bug took 2–4 hours. I wrote up a design change for one traceable load-event ID end to end, with a drop metric at every hop, and got the Visibility, Ocean and Flexport OS managers to slot the stories. I also built the Datadog monitoring checklist for every connectivity point. **R** Blocker-bug MTTR fell by 50%, and we were monitored at every connectivity point before the customer-ready date.

Land: 2–4 hours to locate → MTTR −50%. **Don't say:** "I built the trace", since three teams built it. **Drawn from:** R5.

3-level follow-up. L1 "Did the engineering managers resent it?" → One pointed out the downstream system couldn't carry the ID yet, which was right, so it became a dependency with its own design doc. L2 "How was the 50% measured?" → Blocker troubleshooting before and after, from my own one-pager. L3 "What would you do earlier?" → Trace and monitors in sprint one, which was our own retrospective lesson.

AIQB: OWNERSHIP Q2 **"Describe a time when you didn't think you were going to meet a commitment you promised. How did you identify the risk and communicate it to stakeholders? Is there anything you would do differently?"**

Opener: "I committed to a pipeline date and found mid-build that I'd never planned the security review. I told my manager the same day that it was my scoping miss."

S Flexport: automating the weekly business review onto Snowflake, on ticket data with customer content. **T** Hold the commitment without skipping a weekly review. **A** When the access review surfaced, I priced it at two weeks and told leadership the slip was my scoping, because the BRD priced the build and never the review. I kept the manual four-system review running in parallel so the Monday review never skipped, closed the review properly with data classes and access rules, and cut over only at ≤1% parity. **R** Two weeks late on the review, zero review weeks skipped. Security lead time is now a kickoff line on every data program.

Land: two weeks; zero skipped reviews. **Don't say:** "security was slow". **Drawn from:** F2.

3-level follow-up. L1 "How did you find out?" → It surfaced mid-build when the data-access request hit review, and I hadn't scheduled it. L2 "Differently?" → Ask at kickoff whose approval the data needs and how long it takes, and price it in the BRD. L3 "Has it recurred?" → No. My later agent build had access boundaries designed in from day one.

Invent and Simplify

AIQB: INVENT AND SIMPLIFY Q5 **"Tell me about a time when you had a challenging problem or situation that the usual approach wouldn't address. How did you select an alternative approach? What alternative approach(es) did you consider? …"**

Opener: "Everyone wanted a better solver for the carrier bid. The problem wasn't the solver, it was the lane names."

S At Elementum, a client ran a semi-annual carrier bid: hundreds of lanes, dozens of rate files, about six weeks a cycle. **T** I owned requirements, constraint rules and the parallel run. **A** The usual approach was a stronger optimizer. Before choosing, I sat through a full cycle logging hours. Most of the time went to reconciling lane and mode names across carriers. I considered (a) a new solver, (b) manual cleanup staff, and (c) a lane master with explicit constraint rules, and chose (c). The gate was under 5% mismatch against the last two real awards before the tool's output counted, and we parallel-ran it on one region's live bid. **R** The cycle went from about six weeks to about three, and the pilot region finished three weeks early. Other regions adopted it without a mandate (user anchor).

Land: 6 weeks → ~3. **Don't say:** any award savings dollar. **Drawn from:** R2.

3-level follow-up. L1 "Why not both solver and lane master?" → The solver was fine once inputs matched, so a new one would have been cost without benefit. L2 "Why 5%?" → It had to beat planners' trust in the last two awards. L3 "Who disagreed?" → Procurement, late. They were right about two award rules, which I added.

AIQB: INVENT AND SIMPLIFY Q2 **"Describe the most innovative thing you've done and why you thought it was innovative. … What was the problem it was solving? What was innovative about it?"**

Opener: "The most innovative thing I've built is the least flashy: an LLM triage agent that refuses to propose anything it can't cite, at forty cents an incident."

S An independent build: LLM triage for operational incident queues. The naive version cost $25 an incident. **T** Make it an operations tool: accurate, traceable, under $1. **A** The innovation was the discipline, not the model. I built an eval set before the agent (graded historical incidents, where the past must keep passing), a hard rule of no citation, no proposal, and tiered routing with small models on easy classes and expensive calls only on the hard tail. **R** $25 → $0.40 per incident, a 98% cut, with accuracy held and 100% citation coverage. *Pattern, second example:* the trace-ID design on SV (R5).

Land: $25 → $0.40, 100% citations. **Don't say:** that it was deployed at an employer. **Drawn from:** R7.

3-level follow-up. L1 "What's innovative about routing?" → Proving the cheap path on the eval set before moving traffic, which most demos skip. L2 "Why citations?" → An untraceable proposal makes a human redo the work, while a cited one takes thirty seconds to check. L3 "What would break at Amazon?" → Eval upkeep and the data-access review, both of which I'd schedule at kickoff.

Are Right, A Lot

AIQB: ARE RIGHT, A LOT Q7 **"Describe a time when you brought different perspectives together to solve a problem. What types of different perspectives were represented? How did you seek out different points of view? What was the outcome? …"**

Opener: "Each of three engineering teams saw its own hop. I put the product view, the engineering view and the operations view on one seam."

S Shipment Visibility bug bash: data crossing six system layers owned by different teams, with 2–4 hours to locate each blocker. **T** Converge the bug bash and decide how to test 3PL volume safely. **A** I brought engineering managers (what each system can carry), product (what the customer screen must show) and the DRI (customer risk) into one pros-and-cons review. Outcomes: a trace-ID design change across Visibility, Ocean and Flexport OS; a load-event tracing metric; and, for 3PL testing, a production client test account instead of staging, restricted to named clients and lanes, with a kill-switch list and 10% shipment spot checks. **R** MTTR on blocker bugs fell by 50%, and the test ran in production without client disruption.

Land: MTTR −50%. **Don't say:** "staging was useless". The trade-offs were analyzed. **Drawn from:** R5.

3-level follow-up. L1 "Who disagreed on production?" → Risk-averse voices preferred staging, and the kill switches and client/lane restriction were their price for agreeing. L2 "Were they right?" → Partly. Staging was safer, but it couldn't show real 3PL data paths. L3 "Learning?" → Write the "not a goal" list. We explicitly excluded performance and cost-to-serve from that test.

AIQB: ARE RIGHT, A LOT Q4 **"We don't always make the right decision all the time. Tell me about a time when you made a bad decision. What was the impact of that decision? What did you learn? How have you applied what you learned?"**

Opener: "I wrote an availability rule that never let any channel promise stock in transit. It felt safe, and it starved dotcom."

S Sephora's new ISM availability model. My v1 ruleset excluded in-transit everywhere. **T** Fix my own rule without weakening oversell protection. **A** The Aging In-Transit and replenishment reports showed fast movers one day out and conversion lost. I wrote the evidence pack with "this was my rule" in the first line and redesigned it as a per-lane promise window from receiving history, with an on-time floor and a kill switch. I took it back through the same three signatories as a signed change on the normal release cadence. **R** Sellable units restored on qualifying lanes, oversell up zero, kill switch never fired. Applied since: both sides of a rule get a number. The Locus 98% gate and the 0.5% migration tolerance were both set before the event.

Land: oversell +0; kill switch never fired (user anchor). **Don't say:** a lost-sales dollar. There isn't one in the record. **Drawn from:** F3 (use F1 if Ryan heard S4).

3-level follow-up. L1 "Why was it bad, not just conservative?" → I priced one side only. L2 "Why not a quick config change?" → A side config is a second, unsigned truth about "available", the bug the program existed to kill. L3 "How long were you wrong?" → Weeks after go-live, until the reports showed it.

Learn and Be Curious

AIQB: LEARN AND BE CURIOUS Q2 **"Describe a time when you took on work outside of your comfort area. How did you identify what you needed to learn to be successful? How did you go about building expertise to meet your goal? Did you meet your goal?"**

Opener: "I learned LLM engineering by building a production-shaped agent myself, not by reading about it."

S I'm a program leader, and LLM engineering was outside my comfort zone. **T** Build an incident-triage agent good enough for operations: accurate, cited, under $1 an incident. **A** I worked out what to learn from the failure modes: cost, hallucinated actions, regressions. I learned each by building the control for it: an eval set to catch regressions, a citation layer to stop unsupported proposals, and model routing for cost. Stack: Python, FastAPI, LLM tool use, SQLite. **R** $25 → $0.40 per incident, 100% citations, accuracy held. Goal met and exceeded.

Land: −98% per incident. **Don't say:** "I'm an ML engineer". **Drawn from:** R7.

3-level follow-up. L1 "What was hardest to learn?" → Evaluation: what "correct triage" means, graded and versioned. L2 "What would you do differently?" → Freeze eval-set v1 on day one. L3 "How does it apply here?" → The same gate-before-trust logic I'd put on any automation in infrastructure operations.

AIQB: LEARN AND BE CURIOUS Q1 **"Tell me about a time when you realized you needed a deeper level of subject matter expertise to do your job well. What did you do about it? What was the outcome? …"**

Opener: "I'd never landed warehouse robotics. I needed to understand the floor before I could set a gate the robots had to pass."

S Sephora UDC: Locus robots on the retail outbound floor, inside a live WMS conversion. **T** Set a go-live gate I could defend. **A** I ran a three-day Locus workshop (July 29 to August 2, 2024) covering the facility tour, bot map, pick zones, induction rules, the dotcom-versus-retail priority contention, tote types and WMS integration requirements. I had the interfaces unit-tested with simulated data first. From that, I set a 98% pick-accurate throughput bar (99% stretch) before the robots arrived, with manual carts as the planned fallback. **R** Week two measured 94%. About 30% of volume was throttled to carts, zone logic was fixed, and full automation came in week three. The gate worked because I understood what it measured.

Land: 98% bar, 94% in week 2, full automation in week 3. **Don't say:** "the robots failed". **Drawn from:** P3 (program-packet primary).

3-level follow-up. L1 "Why 98%?" → That's where the customer promise lives for retail picks. L2 "Who wanted to push through at 94%?" → The pressure to keep the date. The throttle was a dial we'd already built. L3 "After?" → The bar set before the event became my default (see F3).

Hire and Develop the Best

AIQB: HIRE AND DEVELOP THE BEST Q4 **"Tell me about a time when you provided coaching for a team member. What started the coaching? What was the outcome?"**

Opener: "The most skeptical person on the team was our QA lead. I coached them into writing the gate everyone else had to pass."

S Sephora: a 20-person product, engineering and QA team moving from six-week hybrid cycles to a four-week cadence. VP sponsorship, no mandate. **T** Make the cadence stick, which meant QA had to own quality, not fear it. **A** QA feared their testing window would be crushed. That fear started the coaching. I gave the QA lead the blank page for the definition of done with entry criteria, asked "what would make this safe for QA?", reviewed drafts one-on-one, and had them present it to the team. **R** A four-week cadence in six weeks, critical defects down 50%, and the QA lead's definition of done became the team's gate. Neighbouring teams copied it.

Land: 6 weeks; critical defects −50%. **Don't say:** "my direct report". I had none. **Drawn from:** R9.

3-level follow-up. L1 "What did you actually say to them?" → I asked questions rather than giving a draft, and reviewed in private before the team saw it. L2 "Did it ever not work?" → It took longer than if I'd written it myself, and I'd start with QA earlier. L3 "Raise the bar?" → I'd hire people who write the gate rather than wait for it.

AIQB: HIRE AND DEVELOP THE BEST Q3 **"Give me an example of a time you provided feedback to develop the strengths of someone on your team. Were you able to positively impact their performance?"**

Opener: "On a six-site rollout I made each site's super-user the owner of their own data maps, and fed their reconciliation results back to them every week."

S Elementum: a contract manufacturer's data across six site systems with six definitions of the same fields. **T** Roll out site by site, with the capability living at the site, not with me. **A** Each site named a super-user who owned that site's mapping tables. On the weekly call, the per-site reconciliation report was their feedback loop: where they matched, where they didn't, why. I built on their strength, local knowledge of what a field really meant at their site, and let them correct the maps where the source's real numbers disagreed with its documentation. **R** Six sites in three waves over about five months, and the super-users kept the maps after go-live (user anchor). *Honest frame:* these were site owners, not direct reports.

Land: 0 → 6 sites in three waves (user anchor). **Don't say:** headcounts or promotions. None are in the record. **Drawn from:** R3.

3-level follow-up. L1 "Example of the feedback?" → Where the weekly reconciliation showed a mismatch, the super-user traced whether their map or the source documentation was wrong, and fixed their own map. I reviewed it with them; I didn't do it for them. L2 "Did anyone struggle?" → The legacy site, with no change-data-capture, took six weeks against three, so we gave them snapshot-diffing. L3 "People management?" → I've led through programs, not org charts, and I say so.

Insist on the Highest Standards

AIQB: INSIST ON THE HIGHEST STANDARDS Q6 **"Describe the most significant continuous improvement project that you led. What was the catalyst to this change and how did you go about it? What was the outcome?"**

Opener: "The catalyst was simple: on a thirty-track warehouse program, status by opinion would have drowned us. So I made readiness a count."

S Sephora UDC: two warehouses into one omni building, about a dozen systems, 60-plus stakeholders, about thirty build tracks. **T** Make every go-live call off counted rows. **A** One consolidated issues log with one grammar (status, priority, operation, area, owner, date), no side lists, and a weekly dedupe. Blocker and critical rows (441 of 836, 52.8%) led every review, and each red had a name and a date. A parking-lot status meant "decided not now", in writing. Then I made the log the gate: both go-lives read burn-down curves. **R** 836 rows, 85.4% closed. The June 2025 spike of 271 burned down 65, 21, 14, 4. Zero customer-facing incidents at both go-lives.

Land: 836 rows; 85.4% closed; 271 → 4. **Don't say:** "836 bugs". Only 115 were bugs. **Drawn from:** R8.

3-level follow-up. L1 "What made it continuous, not one-off?" → The weekly read and dedupe, run for the whole program. L2 "What did the data change?" → An owner's queue growing two weeks running triggered a resourcing conversation with evidence. L3 "Differently?" → Monthly-shape view from the first hundred rows.

AIQB: INSIST ON THE HIGHEST STANDARDS Q1 **"Describe a time when you refused to compromise your standards around quality/customer service, etc. Who was your customer? What was the result?"**

Opener: "In week two the robots measured 94% against a 98% bar I'd set before they arrived. I throttled them rather than lower the bar."

S Sephora UDC: Locus robotics on retail outbound inside a live WMS conversion. **T** Protect the retail customer's pick accuracy while landing automation. **A** I'd set the bar at 98% pick-accurate throughput (99% stretch) before the bots arrived, with manual carts as a planned fallback. When week two read 94%, there was pressure to keep full volume on the robots. I moved about 30% of volume back to manual carts, got the zone-logic fix in, and re-measured. **R** Full automation in week three, and the customer was never the test. The customer was the stores and dotcom shoppers downstream of those picks.

Land: 98% bar; 94% → throttle ~30% → full automation in week 3 (record). **Don't say:** "the robots failed". The gate worked. **Drawn from:** P3.

3-level follow-up. L1 "Who pushed to keep volume on?" → The schedule pressure. My answer was that the throttle was a dial we'd already built. L2 "What did the throttle cost?" → Manual labour for about a week on 30% of volume. L3 "Would you set 98% again?" → Yes, because that's where the customer promise lives.

Think Big

AIQB: THINK BIG Q1 **"Tell me about time when you were working on an initiative or goal and saw an opportunity to do something much bigger or better than the initial focus. Did you take that opportunity? Why or why not? What was the outcome?"**

Opener: "The ask was one Austrian site's carrier problem. I built it as a method for twenty-five locations."

S Elementum: the client's Austria site couldn't measure on-time delivery, with 40% reported data quality. **T** Initial focus: fix Austria. **A** I designed everything to travel. The rules engine was keyed on ship-from, ship-to, mode, volume, Incoterm and SLA. The savings method was co-written with Finance and per-shipment auditable. The data gate was 99% before scale, and there was a one-page runbook with regional super-users. Then I ran Austria as the pilot on the ugliest carrier mix and let its numbers make the scale call. **R** The pilot went live inside a month at about 97% on-time accuracy and about $500K a year. The rollout covered 25 European locations, with about $8M of validated freight savings, and Finance reused the method.

Land: ~$8M across 25 European locations. **Don't say:** "400-plus lanes"; "$1M". **Drawn from:** R1.

3-level follow-up. L1 "Why not scale faster?" → The engine waits for the data. L2 "How did you track adoption?" → Sites added by one rules file plus one data-quality cycle (user anchor), and Finance-audited claims per site. L3 "What would you change?" → Settle the audit-ledger format during the pilot.

AIQB: THINK BIG Q4 **"Tell me about a time when you drove adoption for your vision/ideas. How did you know your vision/idea was adopted by others? How did you drive adoption for your vision/ideas? How did you track adoption? …"**

Opener: "I knew the cadence was adopted when teams I'd never talked to copied it."

S Sephora: a 20-person team on six-week hybrid cycles, missing high-value stories. **T** Vision: predictable four-week delivery, owned by the team. **A** I drove adoption through the people, not a mandate: one-on-ones across functions, a named influencer in each, and the most skeptical person (QA) writing the definition of done. I tracked it with one chart, published every cycle, showing stories reaching QA on time and what moved and what didn't. **R** A four-week cadence in six weeks, critical defects down 50%. Adoption showed in two ways: the team kept it without a mandate, and neighbouring teams copied it.

Land: 6 weeks; −50% critical defects; copied by neighbours. **Don't say:** "I rolled agile out to the org". **Drawn from:** R9.

3-level follow-up. L1 "How do you know it wasn't the VP's authority?" → She gave air cover, not orders, and the team chose it every Monday. L2 "What didn't get adopted?" → My own drafts. The team's version replaced them. L3 "Differently?" → Start with QA.

Bias for Action

AIQB: BIAS FOR ACTION Q8 **"Tell me about a time when you were able to remove a serious roadblock/barrier preventing your team from making progress. What was the barrier? How were you able to remove it? What was the outcome? …"**

Opener: "The roadblock was two programs fighting over one engineering team. I removed it by putting both on one ruler inside a week."

S Flexport, 2022: leadership pulled a strategic go-live forward a month and shared my engineers. About $200K a month per DC of savings was at stake. **T** Keep both programs moving with one team. **A** The barrier was context-switching and priority arguments. I set up one RAID log, one 45-minute daily forum and one ruler: true blocker, non-blocker, or requirements gap. Within about a week most "blockers" were reclassified. True blockers went to a two-pass onsite/offshore war room with written handoffs, and escalations went up the same day as priced choices. **R** The compressed program landed inside four weeks, mine held its original date, and the business adopted on time.

Land: ~$200K/month per DC; inside 4 weeks. **Don't say:** "$1M per DC" first. Quote the rate. **Drawn from:** R6.

3-level follow-up. L1 "What did you personally rule on?" → I chaired the forum's classification calls and framed the escalations. L2 "What if leadership had refused a priced choice?" → Then the floor (blockers to agreed level) would have decided go/no-go, not the calendar. L3 "Cost?" → The team's calendar, which I named.

AIQB: BIAS FOR ACTION Q4 **"Tell me about a time when you had to gather information and respond immediately to a situation. What was the outcome? Would you have done anything differently?"**

Opener: "Cyber Monday, 4 AM: the VP called because backorders were piling up with inventory on the shelf."

S Manhattan Associates: I ran distribution OMS programs for a major retailer, 3 DCs and 100 stores, and owned holiday monitoring. About $6M of sales was on the line. **T** Stop the bleeding before the morning surge. **A** My own alerts named the signal: backorder variance on critical order types. I ran one bridge, one log and one decision list, every call timestamped. The noise said capacity. The trace said the allocation scheduler had stopped silently at the daylight-saving change. I triaged on promise-at-risk first, rebalanced allocation across DCs, confirmed with the client's business team, restarted, and sent status every two hours until 9 PM. **R** Fill rate held at 99% against a 95% goal, and about $8M of the day's sales was protected. The post-mortem added a scheduler heartbeat, so silence itself became an alarm.

Land: 99% vs 95%; $6M on the line → ~$8M protected (record). **Don't say:** $8M as both "at risk" and "protected". **Drawn from:** P6 (also with Josyula as S13; tell this incident-command angle).

3-level follow-up. L1 "What did YOU decide at 4 AM?" → Promise-at-risk first, and the DC rebalance, signed in the log. L2 "Differently?" → The heartbeat should have existed before peak. Silence wasn't monitored. L3 "After?" → The next peak ran boring.

Frugality

AIQB: FRUGALITY Q4 **"Tell me about a time when you had to make tradeoffs between quality and cost. How did you weigh the options? What was the result? Would you have done anything differently?"**

Opener: "Twenty-five dollars an incident was accurate but unaffordable. I got it to forty cents without giving up accuracy, and the eval set is how I proved it."

S My LLM triage agent: accurate, but at $25 an incident. **T** Under $1 an incident with accuracy held. **A** I weighed it per class, not overall. The eval set showed which incident classes a small model handled at equal accuracy. Those went to cheap models, and expensive calls were reserved for the hard tail. Every routing change had to keep the eval set passing, so cost could only fall where quality was proven. **R** $25 → $0.40, −98%, accuracy held, 100% citations.

Land: −98%. **Don't say:** "no trade-off at all". The trade was build time for the eval set. **Drawn from:** R7.

3-level follow-up. L1 "Where did you accept lower quality?" → Nowhere on the eval set. The cost was upfront engineering time. L2 "Differently?" → Version the eval set on day one. L3 "At scale?" → Eval upkeep becomes its own program line.

AIQB: FRUGALITY Q3 **"Tell me about a time when you didn't have enough resources to do something you felt was important but found a creative way to get it done anyway. What was the situation? What other options did you consider? …"**

Opener: "One site couldn't fund the system upgrade we needed, so we detected changes ourselves by diffing their flat-file snapshots."

S Elementum: a six-site contract-manufacturer rollout. The third-wave site ran a legacy system with flat files and no change-data-capture. **T** Get it to planning grade without a budget for an upgrade. **A** Options: (a) wait for the site to upgrade, (b) manual weekly reloads, (c) snapshot-diffing on the flat files to detect changes. I chose (c), scheduled the site last with buffer, and kept the same reconciliation gate as every other site. **R** Live in six weeks against a three-week average, absorbed in the plan, and all six sites were live in about five months (user anchor).

Land: 6 weeks vs 3 (user anchor). **Don't say:** a dollar figure. None in record. **Drawn from:** R3.

3-level follow-up. L1 "Who built the diff?" → Data engineering, from my requirement. L2 "Risk?" → Missed intra-day changes, acceptable at planning cadence. L3 "Differently?" → Sample reconciliation before writing the spec.

Earn Trust

AIQB: EARN TRUST Q5 **"Tell me about a time when your team's goals were out of alignment with another team you relied on in order to meet your goal. How did you work with the other team? Were you able to achieve your goals?"**

Opener: "Customs Platform and our forwarding app were both building export customs. Fifteen work items, and two teams who each thought some were theirs."

S Flexport, 2022: the 2023 plan had 15 work items, 12 export and 3 import, overlapping between Customs Platform and the FF App. **T** Get one owner per item before build planning, since we relied on Customs for licensed filings. **A** I proposed a principle (customs owns planning and filing, the app owns moving cargo onto a vessel) and one table per item: user group, exists in core, proposed owner, notes. We worked it row by row in two sessions, decided on qualification (only licensed brokers file AES and ISF), and put APIs at the seams: create, amend, query and delete for AMS. **R** On Nov 3, 2022, the FF App took ownership of all export items except ISF filing, AMS and AES ran through Customs APIs, and both teams shared the cost-to-serve goal.

Land: 15 items; one owner each except ISF. **Don't say:** "I wrote it" unless confirmed VERIFY. **Drawn from:** R4.

3-level follow-up. L1 "What did you give up?" → EAD stayed offline for now, and China documents may be revisited in 2025. L2 "Who was hardest?" → On AES and ISF, the right business owner wasn't in session one. L3 "Did delivery follow?" → The doc records the agreement, not a later delivery metric, so I don't claim one.

AIQB: EARN TRUST Q4 **"Give me an example of a time when you were not able to meet a commitment. What was the commitment and what were the obstacles that prevented success? What was the impact to your customers/peers and what did you learn from it?"**

Opener: "I committed a training sign-off date on our WMS upgrade and missed it by a week and a half, on a task everyone had called three days."

S Sephora RP → JDA: two DCs in eight months. Training sign-off was a go-live exit criterion, gated on hundreds of RF guns, printers and workstations converting. **T** Hold the training date. **A** The device task was estimated at 3–4 days, and I planned it as low effort. I scheduled a device test before training anyway, and the upgrade script failed on new hardware. With key people on COVID night shifts, the manual conversion couldn't finish in time. I told leadership directly: training slips 1.5 weeks, and it's my miss. **R** Training slipped 1.5 weeks. The program still landed on time and about 3% under budget. Learned: impact, not effort, decides a task's attention, so every cross-functional dependency now gets a what-if plan.

Land: hundreds of devices; +1.5 weeks; ~3% under budget (record). **Don't say:** an exact device count. **Drawn from:** P7 (G2) in the program packet.

3-level follow-up. L1 "Whose estimate?" → The devices team's, but accepting an effort estimate as a risk assessment was mine. L2 "Impact to peers?" → SME windows were scarce, so trainers re-planned. L3 "Applied since?" → Mock cutovers 7.0% → 1.8% → 0.3% against a 0.5% tolerance.

Dive Deep

AIQB: DIVE DEEP Q6 **"Have you ever created a metric that helped identify a need for a change in your department? What was the metric? Why did you create it? How did this and other information influence change? What was the outcome of the change?"**

Opener: "I created a drop ratio at every hop between six systems: events arriving divided by events sent. It showed exactly where data died."

S Flexport Shipment Visibility: data crossed six system layers, and locating a blocker took 2–4 hours. **T** Make data loss visible per system so the right team fixed the right thing. **A** I defined the load-event tracing metric at each hop (Visibility → Ocean → Flexport OS → Client App): events in the originating system ÷ events in the destination, and events dropped ÷ events planned. It required a unique trace ID end to end, which I proposed as a design change. For the 3PL test I made this ratio the success metric and wrote down what wasn't a goal: performance and cost-to-serve. **R** Blocker-bug MTTR fell by 50%. Engineering investigated specific dropped or erroneous trace IDs instead of guessing, and product spot-checked 10% of shipments.

Land: 2–4 hours → MTTR −50% (record-from-docs). **Don't say:** a drop-rate percentage. None is recorded. **Drawn from:** R5.

3-level follow-up. L1 "Why per hop, not end to end?" → End to end tells you that data died, and per hop tells you where. L2 "What other info mattered?" → The Datadog monitors for P95 event age and stream blockage per connectivity point. L3 "What would you do earlier?" → Put the metric in sprint one.

AIQB: DIVE DEEP Q5 **"Tell me about a specific metric you have used to identify a need for a change in your department. Did you create the metric or was it already available? How did this and other information influence the change? What was the outcome of this change?"**

Opener: "The available metric said data quality was 40%. I measured it myself at shipment grain, got 55%, and that changed the whole program."

S Elementum: the client's Austria transportation team. The reported data-quality metric of about 40% was used to blame carriers. **T** Find out what was actually broken. **A** I didn't trust the available metric's grain. I pulled three years of shipment history in SQL, measured quality per field class at shipment grain (about 55% true), and counted defects by class: missing rates, wrong-grain lanes, five spellings of carrier names, brochure transit times. That reframed the change from "penalize carriers" to "fix measurement", with a 99% gate and a weekly burn-down per defect class. **R** 99% within a quarter, Austria pilot about 97% accurate and about $500K a year, and about $8M validated across 25 European locations.

Land: 40% reported → 55% true → 99%. **Don't say:** "two months" or "three months". Say "within a quarter". **Drawn from:** R1.

3-level follow-up. L1 "Why was 55% higher than 40%?" → The report's grain penalized records that were usable at shipment grain, but neither number could hold a carrier to an SLA. L2 "How did you know the defect classes were the right ones?" → Each was counted and burned down weekly, and quality moved as they closed. L3 "Who owned each class after?" → Named owners, then regional super-users.

Have Backbone; Disagree and Commit

AIQB: HAVE BACKBONE; DISAGREE AND COMMIT Q3 **"Often, we must make decisions as a group. Give me an example of a time you committed to a group decision even though you disagreed. What factors led you to commit to the decision? Would you make the same decision now?"**

Opener: "Leadership decided to pull another go-live forward a month and take my engineers to do it. I disagreed, said so with the cost attached, then committed completely."

S Flexport, 2022: two programs, one shared team, an immovable new date. **T** Make the decision work, including for my own program. **A** I disagreed visibly but briefly, framing it as a priced choice rather than a complaint. What made me commit: about $200K a month per DC was at stake, so the pull-forward was probably right. Then I committed with mechanisms, not words: one RAID log, one 45-minute daily forum, one blocker ruler and a two-pass war room. **R** The compressed program landed inside four weeks and mine held its original date. Would I do it again? Yes. Disagree-and-commit only works if the disagreement was visible first.

Land: inside 4 weeks; both landed. **Don't say:** "I won the argument". I didn't. **Drawn from:** R6.

3-level follow-up. L1 "What did you say in the room?" → "Here's what A costs and what B costs." L2 "Was leadership right?" → On the priced numbers, probably yes. L3 "What did commitment cost you?" → My team's calendar, which I guarded with the 45-minute cap.

AIQB: HAVE BACKBONE; DISAGREE AND COMMIT Q2 **"Describe a time when you took an unpopular stance in a meeting with peers and your leader. What was it? Why did you feel strongly about it? What did you do? What was the outcome?"**

Opener: "The room had decided the carrier bid needed a better optimizer. I said it wasn't an optimization problem."

S Elementum: the client's semi-annual carrier bid, about six weeks a cycle. The expected fix was a stronger solver. **T** I owned requirements, and buying the wrong thing would have wasted a cycle. **A** I felt strongly because a better solver on mismatched lane names gives faster wrong answers. Instead of arguing, I asked for one cycle to observe, logged the hours by activity, and brought back the time audit: most of the six weeks was reconciliation. I proposed a lane master, explicit constraint rules, a less-than-5% match gate against the last two awards, and a written rejection of a black box. **R** The cycle went from about six weeks to about three, and the pilot region finished three weeks early. Other regions adopted it without a mandate (user anchor).

Land: 6 → ~3 weeks (user anchor). **Don't say:** award savings. **Drawn from:** R2.

3-level follow-up. L1 "How did your leader react?" → They agreed to one observed cycle, and the audit did the arguing. L2 "What if the audit had said optimization?" → Then I'd have built the solver, since the audit was the test of my view, not decoration. L3 "Miss?" → Procurement late.

Deliver Results

AIQB: DELIVER RESULTS Q1 **"Give me an example of a time when you were able to deliver an important project under a tight deadline. What sacrifices did you have to make to meet the deadline? How did they impact the final deliverable? What was the final outcome?"**

Opener: "Two warehouses into one live omni building, with a peak-season freeze we couldn't move. I gave up the single go-live to make the dates."

S Sephora UDC: about a dozen systems, 60-plus stakeholders, about thirty build tracks, with an October–December peak freeze. **T** Land it without touching peak or the customer. **A** The sacrifice was scope sequencing. Instead of one big-bang date, PO creation went live in January 2025 and the combined go-live followed in May 2025. I ran the gate chain on counted rows, a 343-task cutover plan of about 815 task-hours, and pre-written rollback triggers: sync below 99% for 4 hours, or p95 above 2× for 1 hour. **R** Zero customer-facing incidents, processing efficiency up 15%. The cost of the sacrifice was a longer hypercare tail across two go-lives.

Land: 0 customer incidents; +15% (record). **Don't say:** "twenty-plus systems"; "fifteen tracks". **Drawn from:** P1.

3-level follow-up. L1 "Who wanted one date?" → The prevailing instinct, because it's simpler to announce. L2 "Did rollback ever fire?" → No, but the triggers were written so nobody debated at 2 AM. L3 "Differently?" → Vendor dependencies on the risk register from day one (F4).

AIQB: DELIVER RESULTS Q2 **"Tell me about a time when you had significant, unanticipated obstacles to overcome in achieving a key goal. What was the obstacle? Were you eventually successful? Knowing what you know now, is there anything you would have done differently?"**

Opener: "Our first mock cutover on the WMS migration came in at 7% reconciliation error against a half-percent tolerance."

S Sephora RP → JDA: two DCs onto a new WMS in eight months, a domain I'd never run. **T** Go live inside the 0.5% reconciliation tolerance I'd published with Finance before mock one. **A** The unanticipated obstacles: mock one at 7.0%, with defect classes in UoM mappings, location-naming drift and open-order status vocabularies, then an RF-device script that failed on new hardware. I ran each defect class to an owner, re-ran the full dress rehearsal, and held the tolerance. Then a 72-hour freeze, and RP kept read-only for 30 days. **R** Mocks went 7.0% → 1.8% → 0.3% against 0.5%, the migration landed in eight months, about 3% under budget, and hypercare exited in week three of four.

Land: 7.0 → 1.8 → 0.3% vs 0.5%; ~3% under (record). **Don't say:** any multiplier; "$2.5M a month" without "my division". **Drawn from:** P2.

3-level follow-up. L1 "Why publish the tolerance before mock one?" → So nobody could renegotiate it once the number was red. L2 "What would you change?" → A what-if on every dependency, including the RF devices. L3 "Who did the fixing?" → 14 core plus 6 DC Ops. I owned the scorecard and the gate.

Strive to be Earth's Best Employer

AIQB: EARTH'S BEST EMPLOYER Q5 **"Tell me about a time when you showed compassion toward someone during a stressful or high-pressure situation. How did you recognize that it was important to demonstrate sensitivity? What did you do to demonstrate compassion? What were the outcomes for that person and the situation?"**

Opener: "When a go-live was compressed onto my team, the easy move was to have everyone work around the clock. I designed it so nobody had to."

S Flexport, 2022: the go-live was pulled forward a month and the same engineers were shared across two programs. **T** Keep 24-hour issue velocity without burning people out. **A** I recognized it in the forum: people were context-switching across two programs and answering at all hours. I set up a two-pass war room, onsite then offshore, with written handoffs, so work moved round the clock but each person worked their own day. I capped the daily forum at 45 minutes and defended the cap. I reclassified non-blockers so engineers stopped firefighting non-bugs, and escalated priced choices upward so the team didn't carry the decision. **R** Both programs landed, the compressed one inside four weeks. I name the honest cost: weeks of two-pass days on the team's calendar.

Land: 45-minute forum; nobody worked 24 hours. **Don't say:** "no one was stressed". **Drawn from:** R6.

3-level follow-up. L1 "How did you know it mattered?" → Context-switching between two masters was visible in the forum. L2 "What did people say?" → The handoff notes replaced late-night pings. That is the outcome I can stand behind VERIFY feedback quotes. L3 "Differently?" → Ship the floor guide earlier so DC users weren't stressed either.

AIQB: EARTH'S BEST EMPLOYER Q2 **"Tell me about a time when you advocated for someone who had less influence or seniority than you. How or why did you advocate for them and what was the outcome?"**

Opener: "In our agile conversion, QA had the least influence and the most to lose, so I made sure their working window was protected by design."

S Sephora: a 20-person team moving to a four-week cadence. QA feared compressed testing windows, and peers assumed four weeks meant more work in less time. **T** Make the change safe for the function with the least leverage. **A** I advocated in the design itself. The QA lead wrote the definition of done with entry criteria, so stories couldn't arrive at QA half-ready. I presented it to the VP and the team as QA's gate, not mine, and added a review-status gate in Jira showing where stories sat before QA. **R** A four-week cadence in six weeks and critical defects down 50%. QA's definition of done became the team's standard.

Land: −50% critical defects (record). **Don't say:** a promotion outcome. None recorded. **Drawn from:** R9.

3-level follow-up. L1 "Why QA?" → Their fear was a real design requirement. L2 "Did anyone push back?" → Developers worried about entry criteria slowing them, and the chart showed the opposite. L3 "Differently?" → Start with QA in week one.

Success and Scale Bring Broad Responsibility

AIQB: SUCCESS AND SCALE BRING BROAD RESPONSIBILITY Q1 **"Tell me about a time you worked on a project that had impact beyond your immediate client/customer/organization. How did you approach the potential downstream impact of this work? When in the course of the project did you start thinking about this impact, and what efforts did you make to manage this impact? What were the risks?"**

Opener: "Splitting customs ownership between two Flexport orgs changed who answered to shippers, brokers and ports. So I wrote the downstream effects into the agreement."

S Flexport, 2022: 15 work items for 2023 spanning shippers, external brokers, seaports and US Customs filings. **T** Assign owners without breaking anyone downstream. **A** I thought about downstream impact from the first table: each row named its user group (FlexGo operators, partners, the Flexport Customs team, external brokers) and whether it already existed in core. Risks: a licensed filing rebuilt outside customs, or a broker workflow orphaned. So filings stayed with qualified owners via APIs, EAD stayed with brokers offline until customs could integrate, and portfiling was placed where the seaport dependency lived. Both orgs took on shared cost-to-serve goals. **R** On Nov 3, 2022, the agreement was signed off with deferred items written down with triggers.

Land: 15 items; shared CtS goals. **Don't say:** a downstream metric. None recorded. **Drawn from:** R4.

3-level follow-up. L1 "Biggest risk?" → Regulated filings owned by a team that can't legally file them. L2 "Who was left worse off?" → Operators kept emailing EAD for now, which was written down, with a trigger. L3 "Differently?" → Run import items in parallel.

AIQB: SUCCESS AND SCALE BRING BROAD RESPONSIBILITY Q2 **"Tell me about a time when you decided to change a decision because of unintended consequences for your customers. What was the impact to your customers? What tradeoffs did you have to make? How did your new solution better serve your customers?"**

Opener: "My own availability rule was meant to protect customers from overselling. Its unintended consequence was that dotcom customers couldn't buy stock that was a day away."

S Sephora: my v1 ISM rule barred every channel from promising in-transit units. **T** Reverse my own decision safely. **A** Impact: fast movers were reliably one day from the building but unsellable on dotcom, and conversion was lost (Aging In-Transit and replenishment reports). Trade-off: some oversell exposure in exchange for sellable stock, bounded by a per-lane window from receiving history, an on-time floor and a kill switch. I took it through the same three signatories as a signed change. **R** Sellable units came back on qualifying lanes with oversell up zero, and the kill switch never fired (user anchor).

Land: oversell +0. **Don't say:** a lost-conversion dollar. **Drawn from:** F3 (fallback F1 if Ryan heard S4).

3-level follow-up. L1 "Why not reverse it fully?" → All-in-transit would have recreated oversell on lanes that run late. L2 "How did you size the window?" → Lane σ about 0.3 day gives about half a day at roughly 95% (user anchor). L3 "Lesson?" → Both sides of a rule get a number.

6 · New since this morning — six behavioral questions nobody has asked me yet

NEW

I checked these against both routers. The hiring manager (Tripathy) got CO Q8, CO Q5, Own Q3, I&S Q1, ARL Q1, L&BC Q4, HDB Q9, HSS Q4, HSS Q1, TB Q2, BfA Q1, Frug Q1, Frug Q5, ET Q2, ET Q6, ET Q1, DD Q3, DD Q2, DD Q4, DD Q1, Backbone Q1, Backbone Q4, DR Q3, EBE Q3 and BR Q5. Ryan (Hoerger) was routed by story, not by AIQB number: S19, S8, S8b, S8c, E7, S9, S20 and S6, with S4 as his failure answer. Section 5 above already uses Own Q1/Q2, I&S Q2/Q5, BfA Q4/Q8, TB Q1/Q4, Frug Q3/Q4 and DR Q1/Q2. The six below appear in none of those lists. They lean toward Ownership, Deliver Results, Invent & Simplify, Bias for Action, Think Big and Frugality, and every story is one the HM and Ryan did not hear.

#	AIQB QUESTION	STORY	WHY IT IS NEW FOR ROCKY
N1	Ownership Q4: sacrifice short-term gain for long-term value	R1 routing guide	Elementum transportation was not told to the HM or Ryan. Own Q4 is on no router.
N2	Deliver Results Q5: a goal you didn't think was achievable	R3 six sites, three waves	The HM's Deliver Results question was Q3. S7 at site grain was told to neither.
N3	Invent & Simplify Q6: a novel idea with a big impact	R4 Customs ↔ FFA split	The HM got I&S Q1. The Customs split is in no packet.
N4	Bias for Action Q5: move forward or gather more information	R5 production test account	The HM heard SV as an operating lens. This is the test-environment decision underneath it, so it's the same family from a different angle.
N5	Think Big Q6: improve a process that was working	R2 carrier bid	The HM got TB Q2. S17 was told to neither.
N6	Frugality Q2: completing a project with limited resources	R6 two programs, one team	The HM got Frug Q1 and Q5, from different stories. B3 is FREE.

N1 · Ownership — deferring the savings so the engine wouldn't tender on bad data

AIQB: OWNERSHIP Q4 **"Tell me about a time when you made a hard decision to sacrifice short term gain for something that would create long term value for the business. What was the outcome? Knowing what you know now, would you have done anything differently?"**

Opener: "I held a routing engine that was ready to switch on, and the first savings dollar waited weeks, because the data feeding it was 55% right. That wait is why the program closed at about eight million validated."

S Elementum, a client's Austria transportation site. The routing-guide engine could be configured. Reported data quality was about 40%, and my own SQL census of three years of shipments put it at about 55% true. The sponsor wanted savings now, and he wanted carriers penalized now.

T I owned the analysis, the rules design, the savings method and the scale gate. The short-term gain on the table was turning the engine on and booking early savings. The long-term value was a savings number Finance would sign and a template that could scale.

A I made one rule: the engine waits for the data. Quality had to reach 99% before scale, tracked weekly by defect class with an owner per class. I also gave up the easy headline number. I co-wrote the savings definition with Finance before the first claim: per shipment, contracted guide rate against the counterfactual paid rate, executed only, never "negotiated". I priced the wait for the sponsor: a few weeks of deferred savings, against one visibly wrong tender that would cost the program its credibility with the carriers and the site.

R Data quality reached 99% within a quarter (record). The Austria pilot went live inside a month at about 97% on-time measurement accuracy and about $500K a year (record). That pilot green-lit 25 European locations, which closed at about $8M of validated freight savings (record). Finance reused the method for later claims.

Land: ~55% true → 99% within a quarter, then ~$8M validated across 25 European locations (record). **Don't say:** a lane count, "one million", or a "negotiated" savings figure. Don't say the carriers were blameless either. The scorecard caught real misses later. **Drawn from:** R1. **LPs:** Ownership · Think Big · Insist on the Highest Standards.

TRADE-OFF I MADE

I traded weeks of early, un-auditable savings for a number Finance co-owned. It cost the sponsor's patience and a slower first dollar.

Would I do anything differently? I'd build the audit ledger during the Austria pilot, not alongside the first claims. The first CFO review would have been shorter.

L1 "How did you know the wait was worth it?" → The census showed that wrong-grain lanes and missing rates would route freight to the wrong carrier with confidence. One wrong tender in front of the carriers was the risk I priced.

L2 "Did the sponsor agree?" → Not at first. He wanted penalties now. His own three years of data settled it, so I didn't have to win an argument.

L3 "What did the wait actually cost?" → Weeks of deferred savings at one site. I don't have a dollar figure for the deferral in my record, so I describe it in weeks.

SAFE-SAY: "I deferred the first savings until data quality reached 99%, which took less than a quarter, and the program closed at about eight million validated across twenty-five European locations."

N2 · Deliver Results — the hostile site I didn't believe would make the average

AIQB: DELIVER RESULTS Q5 **"Give me an example of a mission or goal you didn't think was achievable. What was it and how did you help your team try to achieve it? Were you successful in the end? Looking back, would you have done anything differently?"**

Opener: "One of six sites ran a legacy system with flat-file exports and no change-data-capture. I didn't believe it could go live on the same template and the same three weeks as the others, so I said so and planned it last."

S Elementum, a client's manufacturing program. An external contract manufacturer published supply data from six site systems with six definitions of yield, lead time and order status. The goal was planning-grade supply visibility at all six sites (user anchor).

T I owned the ingest requirements, the per-site reconciliation gate, the wave plan and the weekly call with the contract manufacturer. None of the engineers reported to me. The part I didn't think was achievable was the legacy site on the same terms as the others.

A I told the team plainly that the legacy site would not hit the average, and I wrote that into the plan instead of hoping. I sequenced the rollout by risk. Wave 1 was the two cleanest high-volume sites, wave 2 was three sites on parameterized maps, and wave 3 was the legacy site with buffer. For that site we didn't wait for an upgrade it couldn't fund. I wrote the requirement for snapshot-diffing the flat files, and data engineering built it. The gate didn't move. No site went live until its reconciliation report matched the source. Overlapping revisions went to a quarantine queue for a human and were never auto-loaded.

R Six sites live in three waves over about five months (user anchor). The legacy site took about six weeks against a three-week average, and that was absorbed because it was planned last with buffer (user anchor). Zero overlapping revisions were auto-loaded (user anchor). So I was right that it wouldn't make the average, and wrong to think the goal itself was out of reach.

Land: six sites, three waves, about five months; hardest site about 6 weeks vs a 3-week average (user anchor, said as my program counts). **Don't say:** a reconciliation tolerance (none is recorded) or any dollar figure. **Drawn from:** R3. **LPs:** Deliver Results · Are Right, A Lot · Frugality.

TRADE-OFF I MADE

I gave up a single fast cutover for three gated waves. It cost calendar time, about five months, but one bad site could no longer poison trust in all six.

Looking back: I'd run the reconciliation report on a sample before writing the mapping spec, not after. The source's documentation was not the truth; the source system's own numbers were.

L1 "Why say it wasn't achievable instead of just pushing?" → A hidden slip on the last site would have looked like a program failure. A planned one was just a longer wave.

L2 "Why not force the site to upgrade?" → It had no budget and no timeline. Snapshot-diffing gave us change detection without asking them to fund anything.

L3 "What did you personally do on that site?" → I sequenced it, wrote the snapshot-diffing requirement and ran the weekly call where its reconciliation was read. Data engineering built the diff.

SAFE-SAY: "Six sites in three waves over about five months. The hardest site took about twice the average, and the plan already carried that. These are my program counts, and I claim no dollar figure."

N3 · Invent & Simplify — splitting ownership by who is licensed to file, not by who wants the work

AIQB: INVENT AND SIMPLIFY Q6 **"Tell me about a novel idea you had or decision you made that had a big impact on your business. What was novel about it?"**

Opener: "Two tech orgs were both building export customs. Instead of negotiating turf, we split the fifteen work items by one test: who is licensed to file, and who moves the cargo. Then we put APIs at the seams instead of shared code."

S Flexport, 2022. The Customs Platform and Freight Forwarding App (FFA) tech orgs overlapped on customs declaration work, most visibly AMS. The 2023 plan held 15 work items, 12 export and 3 import, with no agreed owner per item (record-from-docs).

T Get one named owner per work item, agreed by both orgs, before 2023 build planning. I was on the FF App program side VERIFY. I'm not on the meeting notes' attendee lists, so confirm my role first. If I can't, answer I&S Q6 with R2 instead. Neither org reported to the other.

A The novel part was the decision rule. Ownership followed qualification and operational ownership, not preference. Only licensed brokers can file AES and ISF, so those stayed with customs or ran through its APIs. Moving cargo onto a vessel is operations, so it sat with FFA. Every work item went into one table: user group, whether it exists in core, proposed owner, open question and engineering conclusion. We worked it row by row in two sessions, on October 26 and November 3, 2022. Where both sides touched an item, we wrote an integration contract instead of sharing code. For AMS, Customs Platform exposes create, amend, query and delete APIs, and FFA collects the filing data and watches for re-file triggers. The "not now" items were written down with a trigger: EAD stays offline with external brokers until customs can integrate, and China export documents may be revisited in 2025.

R Agreement on November 3, 2022: FFA owns every export work item except ISF filing, and AMS and AES run through Customs Platform APIs (record-from-docs). Customs Tech and FFA took shared cost-to-serve goals on AMS (record-from-docs). The impact was that 2023 export planning started with one owner per item and no duplicate build of licensed filing logic. I don't claim a downstream delivery metric, because the document records the agreement, not later results.

Land: 15 work items (12 export, 3 import), one owner each, agreed Nov 3, 2022 (record-from-docs). **Don't say:** "I wrote the proposal" unless confirmed, or any dollar or cycle-time impact. **Drawn from:** R4. **LPs:** Invent & Simplify · Earn Trust · Ownership.

TRADE-OFF I MADE

A split by qualification left ISF and AES open after the first session and kept EAD offline. I accepted a slower, partial answer over a fast split that would have duplicated licensed filing logic.

Failure / learning: The business owner of each regulated item wasn't in session one, so two items stayed open. Now every regulated item's business owner is on the first invite.

L1 "What's actually novel? That's just a RACI." → A RACI records a decision. This was the rule that made the decision, applied the same way to every row, so neither team could relitigate a row on preference.

L2 "Big impact — measured how?" → Every 2023 export item had one owner and one integration contract before build planning. That's what I can source. There's no later delivery metric in the document, so I don't claim one.

L3 "Who pushed back?" → Customs engineering was willing to own AES and ISF filing but needed their business team to decide, and preferred API-only integration. They were right, so we took the API route.

SAFE-SAY: "Fifteen work items for 2023, twelve export and three import. On November 3rd, 2022, we agreed that the app owns every export item except ISF filing, with AMS and AES through Customs Platform APIs. That's what the ownership proposal and its meeting notes record."

N4 · Bias for Action — a production test account instead of waiting a year for organic volume

AIQB: BIAS FOR ACTION Q5 **"Give me an example of when you had to make an important decision and had to decide between moving forward or gathering more information. What did you do? What was the outcome? What information is necessary for you to have before acting?"**

Opener: "We could wait for 4PL clients to grow organically until the end of the year and learn slowly, or test 3PL volume in production now. We moved forward with a client test account in production, 300 containers to start, and a kill-switch list in hand."

S Flexport Shipment Visibility, after launch. To find root causes of data inaccuracies we needed volume, and organic 4PL client growth would not supply it before the end of the year (record-from-docs, 3PL one-pager p1–2). The choice was between testing in staging and testing in production.

T As TPM, run the staging-versus-production decision with product and engineering, then make the chosen path safe for live clients.

A I ran a pros-and-cons analysis of staging versus production with product and engineering, and the team decided on a client test account in production (record-from-docs). Then I bounded the blast radius before anything ran. We started with 300 containers, Trans-Pacific lanes first and then all lanes, using Crux-only clients' data so 4PL and 3PL could be compared (one-pager p6–7). Testing was restricted to named client and trade-lane combinations. I built the list of switches to turn off, such as the Ocean Visibility subscription kill switch and the Trade Graph extract switches. The end-to-end monitoring from the launch stayed in place, and we reused the holiday support framework for blocker and critical issues (p5–6). The success metric was written before the test, as load-event tracing at each hop, along with what was not a goal: performance and cost-to-serve. Dependencies carried owners and dates. The trace-ID stories were due 4/2, and SOA ingestion was blocked on an Infosec ticket. The test was planned to start in the second week of April (p3–4).

R The decision and its controls are recorded in the one-pager (record-from-docs). The trace-ID stories were on track on the Visibility and Ocean teams, and the Flexport OS design doc was complete. The trace-ID change this test measured cut blocker-bug MTTR by about half (record-from-docs). The 3PL test's own final readout is not in my sources VERIFY, so I don't quote one.

Land: 300 containers, Trans-Pacific first, kill switches listed before go (record-from-docs). **Don't say:** a 3PL test accuracy result or "we found X% dropped". Neither is recorded. Don't say "I decided alone". The doc says the team concluded. **Drawn from:** R5. **LPs:** Bias for Action · Are Right, A Lot · Ownership.

"What information do you need before acting?" → Four things: is it reversible (the kill switches), is the blast radius bounded (named clients and lanes, 300 containers), is success written down (the tracing ratio), and is what's out of scope written down (performance and cost-to-serve). If those four exist, more information is a delay, not a safeguard.

TRADE-OFF I MADE

Production carries client risk that staging doesn't. I accepted it in exchange for real data volume now rather than at year end, and paid for it with switches, scoping and support coverage.

Failure / learning: Our own retrospective says monitoring came too late in the lifecycle and test estimates were too optimistic (Path to 4PL p2). I now put the monitors and the kill-switch list in the first sprint.

L1 "Why not staging?" → Staging couldn't give us real 3PL volume or the real data flowing between systems, and the whole point was to find where real events dropped.

L2 "What would have made you stop?" → Any disruption to a live client. The kill-switch list and the named client and lane scope existed so we could stop without a deploy.

L3 "Did it work?" → The decision and the controls are in writing, and the trace-ID change behind the test cut blocker MTTR by about half. I don't have the 3PL test's own final readout in my sources, so I won't invent one.

SAFE-SAY: "We chose a client test account in production over waiting for organic volume. We started with 300 containers on Trans-Pacific lanes, with the kill switches listed before we began. That's in my one-pager; the test's final readout isn't, so I don't quote one."

N5 · Think Big — the bid process worked; I questioned why it took six weeks

AIQB: THINK BIG Q6 **"Tell me about a time when you thought differently to improve a process that was working. What assumptions did you have to question? How did you evaluate if the change improved the process? Knowing what you know now, would you do anything differently?"**

Opener: "The carrier bid worked. It produced awards twice a year. It also took six weeks, and everyone assumed a better optimizer was the fix. I questioned that assumption by timing the work, and most of the six weeks was reconciliation."

S Elementum, a client's transportation team. A semi-annual carrier bid across hundreds of lanes and dozens of rate files took about six weeks a cycle (user anchor). It worked, so there was no burning platform, only a request for a better solver.

T I owned the requirements, the constraint rules and the parallel run. The award belonged to transportation and procurement.

A I questioned three assumptions. First, that it was an optimization problem. I sat through one full cycle and logged hours by activity, and the time went to reconciling lane names and mode codes. Second, that a better solver should be a black box. I required scenario comparison as a first-class output. Third, that six weeks was simply the cost of the bid. I specified a lane master, wrote every constraint as one readable rule and set a trust gate: the tool had to match the last two real awards within 5% before its output counted. We parallel-ran one region's live bid on both paths, then added a weekly rate-file refresh with a stale-data banner after 10 days.

R I evaluated it with the same team on the same kind of bid, against my own time audit of the previous cycle. The cycle went from about six weeks to about three, the pilot region finished three weeks early, and other regions adopted it without a mandate (all user anchor). No award dollars are claimed.

Land: about 6 weeks → about 3; 5% gate against the last two awards (user anchor). **Don't say:** award savings, "the optimizer saved $X", or "50% faster" as if audited. Say "about half". **Drawn from:** R2. **LPs:** Think Big · Dive Deep · Invent & Simplify.

TRADE-OFF I MADE

I spent one full bid cycle observing before building anything, and I didn't ship the solver everyone wanted. It cost some impatience from a team that wanted a tool.

Would I do anything differently? I'd bring procurement in at the time audit. They owned two award rules I had to retrofit before the second region.

L1 "Why touch a working process?" → It worked by consuming six weeks of analysts twice a year, and the hours were going to data hygiene, not decisions.

L2 "Why 5%?" → It had to beat the planners' trust in the last two awards. Below 5% mismatch they stopped re-keying, and above it they would have kept the spreadsheet running beside it.

L3 "How do you know it wasn't just a quiet cycle?" → Same team, same kind of bid, measured against my own audit of the previous cycle. These are program counts, not an audited report, so I say "about half".

SAFE-SAY: "About six weeks to about three, with the pilot region three weeks early. Those are my program counts. The award dollars belonged to transportation and procurement, so I don't claim them."

N6 · Frugality — two programs landed on one shared team with no new headcount

AIQB: FRUGALITY Q2 **"Describe a time when you had to get a project or initiative completed with limited resources. How did you approach the situation? What was the impact? Knowing what you know now, would you have done anything differently?"**

Opener: "Leadership pulled a go-live forward a month and moved half my engineers onto it. I didn't ask for headcount. I made one team's time count twice."

S Flexport, 2022. A strategic go-live was pulled forward a month onto engineers shared with my own program, with about $200K a month per DC of savings at stake (record). There was no added headcount, and the date wasn't mine to set.

T Land both programs, without the floor rejecting the compressed one, on the team I already had.

A The scarce resource was engineering attention, so I cut what was wasting it. One RAID log and one 45-minute daily forum for both programs, so nobody served two masters. One ruler for every issue: a true blocker, a non-blocker, or a requirements gap dressed as a blocker. Most "blockers" were reclassified within about a week. True blockers went through a two-pass war room, onsite and then offshore, with a written handoff, which gave round-the-clock coverage without anyone working round the clock. Escalations went to leadership as priced choices ("A costs this, B costs that"), so decisions came the same day. Go/no-go read the true-blocker burn-down against a floor agreed in advance.

R The compressed program landed inside four weeks and mine held its original date (record). The business adopted on time, protecting the roughly $200K a month per DC (record). The cost was the team's calendar, and I said so out loud.

Land: ~$200K/month per DC at stake; landed inside four weeks with no added headcount (record). Quote the monthly rate before the ~$1M-per-DC cumulative framing, if you use that at all. **Don't say:** "saved $1M" as a booked result. **Drawn from:** R6. **LPs:** Frugality · Deliver Results · Bias for Action.

If R6 was already told for Bias for Action: pivot to R3, where snapshot-diffing replaced a legacy upgrade the site couldn't fund, and give the six-sites, five-months frame.

TRADE-OFF I MADE

I didn't ask for headcount, which would have arrived after the date anyway. In exchange I spent my own time on a heavier operating rhythm and guarded the 45-minute forum so it didn't grow.

Would I do anything differently? I'd write the ten-step quick-reference guide for the DC floor first, not the week before cutover. Under compression, documentation dies first, and it was the cheapest fix I had.

L1 "Why not ask for contractors?" → They would have been onboarded after the date. The constraint was attention, not bodies.

L2 "Isn't relabeling blockers just optimism?" → No, it's triage. The proof was the true-blocker count burning down to the agreed floor, and go/no-go read that count.

L3 "What did it cost people?" → Two war-room passes a day for weeks. The onsite-offshore split with written handoffs is how nobody worked twenty-four hours.

SAFE-SAY: "About two hundred thousand dollars a month per DC was at stake. Both programs landed on one shared team with no new headcount: the compressed one inside four weeks, and mine on its original date."

7 · Questions to ask Rocky, source trace, VERIFY checklist NEW

Four questions to ask Rocky — infrastructure and program specific

These are new: none of them repeat a question planned for the HM or Ryan. Ask two, and keep two in reserve. Each one has a reason, what to listen for, and the story to bridge to if he opens the door. Don't claim knowledge of his org. Ask, then listen.

> ASK 1 · LAUNCH READINESS **"When a new building or a major retrofit goes live, what's the one readiness signal your team trusts most, and who owns the vendor dependencies on that critical path?"**
>
> **Why:** It's the question a launch program lives or dies on, and it shows I think in gates, not dates. **Listen for:** whether vendor dependencies sit on a risk register with triggers, or as tasks with dates. **Bridge:** F4. "I learned that one the hard way on a robotics retail track. A vendor dependency carried as a task slipped three weeks. Now it's a risk with a trigger date and an owner on both sides."

> ASK 2 · WHERE SCHEDULES SLIP **"Across your launch portfolio, where do schedules slip most often: construction and site readiness, equipment and vendor delivery, or systems integration and testing? How early does your mechanism catch it?"**
>
> **Why:** It tells me where the TPM in this role would earn their keep. **Listen for:** integration and test, which is my strongest ground, versus physical construction, where I'd be learning. **Bridge:** R5 or N4. "On a six-system platform, the slip hid in the seams between systems, so the first thing I'd build is a trace across them and a monitor per connection point."

> ASK 3 · THE WEEKLY READ **"In your weekly review for a launch program, does the room read curves or colors, and which input metric has a single-threaded owner who can move it?"**
>
> **Why:** It's a mechanisms question a Bar Raiser respects. **Listen for:** burn-down against a pre-agreed floor, owner per input, and how red is escalated. **Bridge:** R8. "On the UDC conversion the go-live gate was the issues-log burn-down, with every red carrying a name and a date, not a RAG color."

> ASK 4 · WHAT TRAVELS TO THE NEXT LAUNCH **"When one launch finishes, what actually travels to the next one: a playbook, a checklist, people, or tooling? Who owns keeping it current?"**
>
> **Why:** It's the Think Big / scale question, and it tells me if the role builds templates or fights fires. **Listen for:** whether lessons become standing controls or stay in retrospectives. **Bridge:** R1. "I designed one site's pilot as the template for twenty-five locations. A new site cost one rules file and one data-quality cycle (user anchor)."

Source trace — every number in R1–R9, F1–F4 and N1–N6

Basis labels: **record** is in Apple v11 or the resume. **record-from-docs** is in a primary document in /tmp/sources. **user anchor** is in GFiber v8 only, with no primary support, so say it as "my program count". **Match** is checked against NUMBERS-RECON canon v2 where it has a row.

NUMBER	STORY	SOURCE FILE	PAGE / ROW	MATCH
DQ ~40% reported / ~55% true	R1, N1, DD Q5	Apple v11 F2	p35	MATCH (recon S10.01)
DQ → 99% "within a quarter"	R1, N1	Apple v11 F2 vs master canon	p36	CONFLICT, resolved (see below)
8–10 hrs/week reconciliation	R1	Apple v11 F2	p35	MATCH (S10.03)
Austria ~97% / ~$500K a year / live inside a month	R1, N1	Apple v11 F2	p36	MATCH (S10.05)
~$8M validated, 25 European locations	R1, N1, TB Q1	Apple v11 F2; resume	p35–36	MATCH (S10.07)
6 weeks → ~3; pilot 3 weeks early; 5% gate; 10-day banner	R2, N5	GFiber v8 S17	p166–167	user anchor (S17: no Apple or primary support)

NUMBER	STORY	SOURCE FILE	PAGE / ROW	MATCH
6 sites, 3 waves, ~5 months; 6 vs 3 weeks; 0 auto-loaded	R3, N2	GFiber v8 S7	p93	user anchor (S7.01–.03); told at site grain
15 work items = 12 export + 3 import	R4, N3	Customs_Platform_Ownership_Proposal	p1	MATCH (read directly)
Oct 26 / Nov 3, 2022; FFA owns all but ISF filing; AMS/AES APIs; shared CtS	R4, N3	same, meeting notes (Appendix B)	p13–14	MATCH. Rajesh not on the attendee lists, so role is VERIFY
6 systems, 9 teams	R5	SV 3PL ingestion one-pager	p1	MATCH
Close to 100 people	R5	Path_to_4PL (c6f2)	p1	MATCH
2–4 hrs to locate a bug (before)	R5, DD Q6	one-pager; Path_to_4PL	p1; p2	MATCH primary (recon paraphrase differs, see below)
Blocker MTTR −50%	R5, N4, Own Q1	one-pager	p1	MATCH
Monitors customer-ready 11/19	R5	State_of_SV_E2E_Connectivity_Monitoring	p1	MATCH
Staging → production; perf tests done	R5	TPM status week of 11/19/21	p2	MATCH
10% spot check; trace stories due 4/2; test April week 2	R5, N4	one-pager	p3–4	MATCH
300 containers; Trans-Pacific first; Crux-only clients; kill switches	N4	one-pager	p5–7	MATCH
~$200K/month/DC; ~$1M/DC; inside 4 weeks; 45-min forum; 10-step guide	R6, N6	Apple v11 B3	p19–21	MATCH (recon §3 B3 line). $1M is cumulative framing
$25 → $0.40, −98%; 100% citations	R7	Apple v11 I1	p41–42	MATCH (canon #78)
836 rows; 714 closed = 85.4%	R8	UDC_Consolidated_Issues_Log; Apple C2	log; p24	MATCH (S2.03, 85.41%)
Blocker 164 + Critical 277 = 441 = 52.8%	R8	Apple C2; log	p24	ROUNDING (52.75%)
IT/WMS 342 · Ops 111 · Locus 86 (10.3%)	R8, F4	Apple C2/C3; log	p24, p26	MATCH (S2.06)
Bugs 115 · req 159 · action 117 · questions 23; spike 271 → 65 → 21 → 14 → 4	R8	UDC log (pandas)	type/status columns	MATCH (canon #44–46, #52)
20 people; 6-week hybrid → 4-week in 6 weeks	R9	Apple v11 D3	p31–32	MATCH (S15.01–.02)
Critical defects −50%	R9	resume	p1	MATCH (S15.03; Apple D3 omits it)
5 users × 10 scenarios; crash 2 days out	F1	Apple v11 G1	p43–44	MATCH (canon #79)
500 containers / 20 min / 99.6%	F1	Apple v11 A2; TPM status 11/19/21	p9–10; p10	MATCH (S11.18)
Security review −2 weeks; ≤1% parity gate	F2	Apple v11 G3 / A3; BRD Zendesk→Snowflake	p45–46	MATCH (S16.07)
σ ≈ 0.3 day → ~0.49-day window ≈ 95%; oversell +0; kill switch unfired	F3	GFiber v8 S4	p66–67	user anchor (S4.02–.03). 1.645 × 0.3 = 0.49 ✓
RTL slipped 3 weeks; UAT window 2/28–3/15; combined go-live May 2025	F4	Apple C2/C3; UDC_LOCUS_Retail_Track	p24–26; status page	MATCH (S2.15). Root cause of the vendor slip: VERIFY
343 tasks ≈ 815 task-hours; rollback sync <99% × 4h, p95 >2× × 1h; +15%	DR Q1 (P1)	Apple v11 C2; resume	p24	MATCH (S2.13, canon #51)
Mocks 7.0 → 1.8 → 0.3% vs 0.5%; 72-hr freeze; RP read-only 30 days; 14 core + 6 DC ops; hypercare exit week 3 of 4	DR Q2 (P2)	Apple v11 D1/D2	p28–31	MATCH

NUMBER	STORY	SOURCE FILE	PAGE / ROW	MATCH
$6M on the line → ~$8M of the day's sales protected; 3 DCs + 100 stores	BfA Q4 (P6)	Apple v11 M1; resume	p40–41	CONFLICT, resolved per recon S13/S14 wording
Hundreds of RF devices; 3–4 day estimate; +1.5 weeks	ET Q4 (P7)	Apple v11 G2	p44–45	MATCH. "Hundreds", never a device count
Locus workshop Jul 29–Aug 2, 2024; 98% bar	L&BC Q1, HSS Q1 (P3)	Apple v11 C3	p25–26	MATCH

Numbers that differed between sources, and how I resolved them

ITEM	WHAT DIFFERED	RESOLUTION USED IN THIS BANK
S10 DQ timeline	Apple F2 says 99% in two months. Master canon says about three, never two.	"Within a quarter", which is true under both. Exact month count is VERIFY.
SV 2–4 hours	Recon §3 paraphrases it as troubleshooting cut *to* 2–4 hrs. The one-pager says the missing ID *caused* 2–4 hrs.	Primary doc wins: 2–4 hrs was the **before**, and the after is "MTTR about half".
B3 value	~$200K a month per DC vs "~$1M per DC protected" in the same chapter.	Rate first. The $1M is used only as cumulative framing and never as booked savings.
M1 $6M / $8M	Apple uses $8M both as revenue at risk and as sales protected.	Recon wording: "$6M on the line … about $8M of the day's sales protected."
Issues-log share	52.8% (Apple) vs 52.75% (recomputed).	Said as "just over half the book".
UDC duration	Apple carries both "fifteen months" and "fourteen months". The log spans 2024-03-17 → 2025-10-01.	R8 says "836 rows from March 2024 to October 2025" and gives no month count.
G2 devices	The Apple source-doc title carries a device count, but canon says "hundreds".	"Hundreds of RF devices" only.
S7 / S17 / S4 figures	GFiber v8 only, with no Apple or primary support.	Labeled user anchor every time. S7 is told at site grain, with no component-level framing.

VERIFY checklist — confirm before Friday 10:00, or say the safe sentence

ITEM	WHERE	SAFE SENTENCE IF NOT CONFIRMED
Rajesh's role in the Customs ↔ FFA split	R4, N3, ET Q5, BR Q1	"The ownership proposal and its meeting notes record the agreement. I'll describe what the document says rather than claim authorship." If the role can't be confirmed, drop R4 and use R2.
Root cause of the Locus RTL vendor slip	F4	"The slip came from a vendor-and-integration dependency. The failure I own is that I carried it as a task, not a risk."
Feedback quotes about the B3 team experience	EBE Q5	"I don't have their words written down, so I'll tell you what I did: one 45-minute forum and nobody working 24 hours."
S10: two months or three to reach 99%	R1, N1, CO Q6	"Within a quarter."
S7 / S17 / S4 figures (user anchors)	R2, R3, F3, N2, N5	"Those are my program counts, not a published report, so I say 'about'."
3PL production-test outcome	N4	"The decision and the controls are in my one-pager. The test's final readout isn't in my notes, so I won't quote one."
B3 $1M cumulative window	R6, N6, BfA Q8	"About two hundred thousand a month per DC. That's the number I'd defend cold."
Did Ryan already hear S4 as the failure answer?	F3, ARL Q4, BR Q2	If yes, lead with F1 or F4 and offer S4 only as "a different angle: the signed-change mechanics".

ITEM	WHERE	SAFE SENTENCE IF NOT CONFIRMED
Did Josyula's packet take D3 or G3?	R9, F2	Tell the Rocky angle only: R9 as Hire & Develop, F2 as the planning control.

Spoken guardrails, one last time: no spoken multipliers. Say 9.7× or "7 to 68". "Sixty-six cents to three cents, about 95%". "About $5M avoided" once at most. "A thousand lanes asked; 597 first". The $2.5M and $1.5M figures only ever with their noun. "Six sites", at finished-goods or site grain. RF devices: "hundreds". "About a dozen systems". The S14 $10M is not savings. The S16 figure over $100K was priced, not saved. Walmart S19–S21 and E1–E12 are design cases. Every number carries its basis word.